INDIAN INSTITUTE OF TECHNOLOGY, JODHPUR

Department of Electronics Engineering

SUMMER RESEARCH INTERNSHIP REPORT

On

Intelligent Electronic Surveillance with Camera-Integrated Drones

Submitted by

Aman Shukla

(2412130309001)

B.Tech – Electronics Engineering

Bharat Ratna Sardar Vallabh Bhai Patel Rajkiya Engineering College, Basti

Internship Duration

1 June 2026 – 20 July 2026

Under the Guidance of

Prof. Binod Kumar

Assistant Professor

Department of Electronics Engineering

Indian Institute of Technology, Jodhpur

CERTIFICATE

This is to certify that **Mr. Aman Shukla**, a student of **B.Tech. in Electronics Engineering** at **Bharat Ratna Sardar Vallabhbhai Patel Rajkiya Engineering College, Basti**, has successfully completed his Summer Research Internship at the **Indian Institute of Technology, Jodhpur** under my guidance.

During the internship, he worked on the project entitled **"Intelligent Electronic Surveillance with Camera-Integrated Drones",** involving the study, development, and implementation of a drone-based surveillance system integrating embedded computing, camera-based image acquisition, and artificial intelligence-based object detection.

The work presented in this report is based on the activities and technical work carried out by him during the internship under my supervision. He has demonstrated sincerity, dedication, and a keen interest in learning and applying concepts related to electronics engineering, embedded systems, computer vision, and deep learning.

This report is submitted as a record of the work carried out during the internship at the **Indian Institute of Technology Jodhpur.**

Prof. Binod Kumar
Department of Electronics Engineering
Indian Institute of Technology Jodhpur

Date: _____

Place: Jodhpur, Rajasthan

DECLARATION

I hereby declare that the internship report entitled **"Intelligent Electronic Surveillance with Camera-Integrated Drones"** presents the work carried out by me during my Summer Research Internship at the **Indian Institute of Technology, Jodhpur**, under the guidance of **Prof. Binod Kumar, Department of Electronics Engineering.**

The work presented in this report is based on my study, practical implementation, experimentation, and observations carried out during the internship period. The report documents the development and evaluation of a camera-integrated drone-based surveillance system, including the integration of embedded computing and camera hardware, deep learning-based object detection, YOLO11n model training and deployment, and Raspberry Pi-based edge inference.

I further declare that the contents of this report have been prepared in good faith and, to the best of my knowledge, accurately represent the work undertaken by me during the internship. Wherever information, concepts, datasets, software, or other resources from external sources have been used, the relevant sources have been appropriately acknowledged and cited.

I understand that any discrepancy or misrepresentation in the information presented in this report is my responsibility.

Aman Shukla
B.Tech. – Electronics Engineering
Bharat Ratna Sardar Vallabhbhai Patel Rajkiya Engineering College, Basti

Date: _____

Place: Jodhpur, Rajasthan

Signature: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the **Indian Institute of Technology Jodhpur** for providing me with the opportunity to undertake my Summer Research Internship and to gain valuable exposure to research and practical engineering work in an academic environment.

I am deeply grateful to my supervisor**, Prof. Binod Kumar, Department of Electronics Engineering, Indian Institute of Technology, Jodhpur**, for his guidance, encouragement, and continuous support throughout the internship. His valuable suggestions and technical insights helped me develop a better understanding of the practical aspects of drone-based surveillance, embedded systems, computer vision, and deep-learning-based object detection. His guidance also encouraged me to approach the project with greater clarity, patience, and a problem-solving mindset.

I would also like to thank the faculty members, research scholars, and staff of the **Department of Electronics Engineering at IIT Jodhpur** for their support and for providing a productive environment in which I could learn, experiment, and work on the project titled **"Intelligent Electronic Surveillance with Camera-Integrated Drones."**

I am thankful to the faculty members of **Bharat Ratna Sardar Vallabhbhai Patel Rajkiya Engineering College, Basti,** for their academic guidance and encouragement, which has provided me with a strong foundation for undertaking this internship.

Finally, I would like to express my heartfelt gratitude to my family and friends for their constant encouragement, support, and motivation throughout the internship. Their support has been a continuous source of strength during the learning, experimentation, and completion of this work.

This internship has been a valuable learning experience for me, allowing me to strengthen my technical knowledge, gain practical exposure to hardware and software integration, and develop a deeper interest in research and engineering applications.

ABSTRACT

The integration of unmanned aerial vehicles (UAVs) with embedded artificial intelligence and computer vision provides an effective approach for intelligent electronic surveillance and aerial monitoring. This internship work, carried out under the project titled **"Intelligent Electronic Surveillance with Camera-Integrated Drones"**, focused on the development and study of a camera-integrated drone platform capable of acquiring aerial visual data and performing AI-based object detection using an edge computing device.

The developed system combines an **S550 hexacopter platform, Pixhawk flight controller, Raspberry Pi 4,** and **Raspberry Pi Camera Module V3** to establish an aerial surveillance framework. The Raspberry Pi was configured as the onboard computing platform for image processing and deep-learning-based inference. The **YOLO11n** object detection model was studied, trained, and fine-tuned using aerial imagery, with the **VisDrone dataset** considered for the development of an object detection system suitable for drone-based surveillance applications. The trained model was subsequently transferred from the training environment to the Raspberry Pi and deployed for inference.

As part of the implementation, the Raspberry Pi environment was configured with the required software libraries and dependencies, including **PyTorch, TorchVision, OpenCV, NumPy, and the Ultralytics YOLO framework**. The deployment workflow included operating system installation and configuration, remote access through SSH and VNC, model transfer using the Secure Copy Protocol (SCP), verification of the deployed model, and execution of YOLO-based inference on the Raspberry Pi. The system was successfully demonstrated to perform object detection on the Raspberry Pi, including detection of a person from camera input with an associated confidence score. Inference timing was also examined to assess the computational performance of the model on the Raspberry Pi CPU.

The work provided practical experience in integrating drone hardware, embedded computing, camera-based image acquisition, deep learning, and edge-based computer vision into a unified surveillance system. The study also highlighted the computational constraints associated with running deep-learning models on resource-limited edge platforms. The developed system establishes a foundation for further development toward real-time aerial surveillance, optimized edge inference, live video processing, autonomous mission execution, and improved integration of onboard AI with drone-based monitoring systems.

KEYWORDS

Intelligent Electronic Surveillance, Camera-Integrated Drones, Unmanned Aerial Vehicle (UAV), Raspberry Pi 4, Raspberry Pi Camera Module V3, YOLO11n, Object Detection, Computer Vision, Edge AI, Deep Learning.

TABLE OF CONTENTS

CERTIFICATE ..... **i**
DECLARATION ..... **ii**
ACKNOWLEDGEMENT ..... **iii**
ABSTRACT ..... **iv**
KEYWORDS ..... **v**
TABLE OF CONTENTS ..... **vi**
LIST OF FIGURES ..... **ix**
LIST OF TABLES ..... **xii**
LIST OF ABBREVIATIONS ..... **xiii**
CHAPTER 1: INTRODUCTION **1**
1.1 Background 1
1.2 Motivation 1
1.3 Role of Drones in Surveillance 2
1.4 Role of Artificial Intelligence 3
1.5 Lightweight Deep Learning and Edge Computing 3
1.6 Project Scope 4
1.7 Report Organization 4–5
CHAPTER 2: PROJECT OBJECTIVES AND REQUIREMENTS **6**
2.1 Project Objectives 6
2.2 Functional Requirements 6
2.3 Hardware Requirements 7
2.4 Software Requirements 8
2.5 Scope and Limitations 8–9
CHAPTER 3: SYSTEM ARCHITECTURE **10**
3.1 Overall System Architecture 10
3.2 Major Subsystems 11–13
3.2.1 Flight Control Subsystem 11
3.2.2 Embedded Vision Subsystem 12–13
3.3 System Block Diagram 14
3.4 Power Architecture 15
3.5 Communication and Control Architecture 16–17
3.6 Overall Working Principle 17
CHAPTER 4: HARDWARE IMPLEMENTATION **18**
4.1 S550 Hexacopter Platform 18
4.2 Pixhawk Flight Controller 18
4.3 Brushless Motors and ESCs 19
4.4 GPS Module 20
4.5 Radio Transmitter and Receiver 20–21
4.6 Raspberry Pi 4 22
4.7 Raspberry Pi Camera Module V3 22
4.8 Payload and Weight Estimation 23
4.9 Final Hardware Configuration 24–25
CHAPTER 5: FLIGHT CONTROLLER CONFIGURATION AND VALIDATION **26**
5.1 Pixhawk Configuration Using Mission Planner 26
5.2 Sensor Calibration 27–29
5.2.1 Accelerometer Calibration 27
5.2.2 Compass Calibration 28
5.2.3 Radio Calibration 28

5.2.4 ESC Calibration 29
5.3 Flight Mode Configuration and Failsafe Settings 30
5.4 Flight Testing 31–32
5.5 Preliminary Flight Validation 33
5.6 Observations and Lessons Learned 33
CHAPTER 6:SOFTWARE AND DEVELOPMENT ENVIRONMENT 34
6.1 Software Environment 34
6.2 YOLO11n Model 34
6.3 Development Environment Setup 34
6.3.1 Raspberry Pi OS Installation and Configuration 34–41
6.3.2 Network and System Configuration 41–43
6.3.3 Python Virtual Environment Setup 43
6.3.4 Deep Learning Library Installation 43–45
6.3.5 Development Environment Verification 45–46
CHAPTER 7: DATASET AND DEEP LEARNING MODEL 47
7.1 Dataset Selection 47
7.2 VisDrone Dataset 47
7.3 Dataset Preparation 48
7.4 YOLO11 Architecture 48–49
7.5 Selection of YOLO11n 49
7.6 Model Training and Fine-Tuning 49–50
7.6.1 Training Environment 50
7.6.2 Initial Training of YOLO11n 50–51
7.6.3 Training Progress and Final Epoch 52
7.6.4 Fine-Tuning of YOLO11n 52–53
7.6.5 Completion of Fine-Tuning 53–54
7.6.6 Validation of the Fine-Tuned Model 54–55
7.7 Transfer of Fine-Tuned Model to Raspberry Pi 55–56
7.7.1 Verification of the Deployed Model 56
7.7.2 Model Inference on Raspberry Pi 56–57
CHAPTER 8: MODEL EVALUATIONAND RESULTS 58
8.1 Evaluation Metrics 58
8.2 Model Performance 59
8.3 Confusion Matrix 59–60
8.4 Training Curves 60–61
8.5 Object Detection Results 61–63
8.6 Analysis of Results 63–64
CHAPTER 9: RASPBERRY PI DEPLOYMENTAND EDGE INFERENCE 65
9.1 Raspberry Pi Camera Module V3 configuration 65–66
9.2 YOLO11n Model Deployment and Inference Setup 66–67
9.3 Inference Pipeline 67–68
9.4 Object Detection on Raspberry Pi 69–70
9.5 Edge Inference Performance and Observations 70–71
CHAPTER 10: COMPLETE SYSTEM INTEGRATION 72
10.1 Integrated System 72–73
10.2 End-to-End Workflow 73–74
10.3 Integrated Hardware Setup 74–75
10.4 End-to-End Demonstration 75–76
10.5 Overall Project Outcome 76
CHAPTER 11: CHALLENGES AND ENGINEERING OBSERVATIONS 77

11.1 Hardware Integration Challenges 77
11.2 Raspberry Pi Connectivity and Remote Access Challenges 77–78
11.3 Model Deployment and Dependency Challenges 78
11.4 Raspberry Pi Computational Constraints 79
11.5 Drone Flight and Validation Challenges 79–80
11.6 Engineering Lessons Learned 80–81
CHAPTER 12: FUTURE SCOPE 82
12.1 Autonomous Flight and GPS Navigation 82–83
12.2 Real-Time Aerial Inference and Live Video 83
12.3 Model Optimization for Edge Deployment 83
12.4 Improved Flight Stability and Reliability 84–85
12.5 Autonomous Surveillance Missions 84–85
12.6 Autonomous Landing, Docking and Wireless Charging 85
CHAPTER 13: CONCLUSION 86
REFERENCES 87
APPENDICES 88–101

LIST OF FIGURES

Figure No.	Figure Title	Page No
Figure 3.1	Overall System Architecture of the Drone-Based Electronic Surveillance System	10
Figure 3.2	Complete Drone Platform Used for the Surveillance System	12
Figure 3.3	Radio Transmitter Used for Manual Flight Control	12
Figure 3.4	Raspberry Pi 4 and Pi Camera Module V3 Module V3 Mounted on the Drone Platform	13
Figure 3.5	Raspberry Pi Camera Module V3 Used for Image Acquisition	14
Figure 3.3	System Block Diagram of the Intelligent Electronic Surveillance System	14
Figure 3.4	3S LiPo Battery (5200 mAh) Used for Powering the Drone Platform	15
Figure 3.5	10,000 mAh Power Bank Used for Supplying Power to the Raspberry Pi 4	16
Figure 3.6	Configuration of Remote Access Services on the Raspberry Pi	17
Figure 4.1	S550 Hexacopter Platform Used for the Development of the Surveillance System	18
Figure 4.2	Pixhawk 2.4.8 Flight Controller Installed on the Surveillance Drone	19
Figure 4.3	Brushless DC Motors Installed on the S550 Hexacopter Platform	19
Figure 4.4	u-blox NEO-M8N GPS Module with Integrated Digital Compass Mounted on the Drone	20
Figure 4.5	FlySky FS-i6 Radio Transmitter Used for Manual Drone Control	21
Figure 4.6	FlySky FS-iA6B Receiver Connected to the Pixhawk Flight Controller	21
Figure 4.7	Raspberry Pi 4 Model B (4 GB RAM) Used in the Proposed Surveillance System	22
Figure 4.8	Raspberry Pi Camera Module V3 Connected to Raspberry Pi 4 via CSI Interface	23
Figure 5.1	Mission Planner Connected to Pixhawk Flight Controller (COM9, 115200 Baud)	26
Figure 5.2	Hexarotor (X) Frame Configuration in Mission Planner	27
Figure 5.3	Accelerometer Calibration Completed Successfully	27
Figure 5.4	Compass Calibration in Mission Planner	28
Figure 5.5	Successful Radio Calibration in Mission Planner	29

Figure 5.6	ESC Calibration Settings in Mission Planner	30
Figure 5.7	Flight mode and failsafe configuration in Mission Planner	31
Figure 5.8	Low-altitude hover during initial flight testing	32
Figure 5.9	Medium-altitude flight above the surrounding area	32
Figure 5.10	Higher-altitude flight demonstrating stable operation	33
Figure 6.1	Selection of the Raspberry Pi 4 device in Raspberry Pi Imager.	35
Figure 6.2	Selection of Raspberry Pi OS (64-bit) in Raspberry Pi Imager	35
Figure 6.3	Selecting the microSD card as a storage device with Exclude System Drives turned on	36
Figure 6.4	User account configuration in Raspberry Pi Imager	36
Figure 6.5	Hostname configuration in Raspberry Pi Imager	37
Figure 6.6	Localisation settings configured in Raspberry Pi Imager	37
Figure 6.7	User account configuration in Raspberry Pi Imager	38
Figure 6.8	Wi-Fi configuration in Raspberry Pi Imager	38
Figure 6.9	SSH authentication configuration in Raspberry Pi Imager	38
Figure 6.10	Raspberry Pi Connect configuration for remote access	39
Figure 6.11	Raspberry Pi ID sign-in and authentication key setup	39
Figure 6.12	Raspberry Pi OS image writing confirmation and storage selection	40
Figure 6.13	Confirmation to Erase and Write Raspberry Pi OS to the SD Card	40
Figure 6.14	Raspberry Pi OS Image Writing Progress	41
Figure 6.15	Raspberry Pi IP Address Identification Using hostname -I Command	41
Figure 6.16	Enabling VNC Through Raspberry Pi Configuration	41
Figure 6.17	Raspberry Pi Configuration – Interface Options	42
Figure 6.18	Selecting VNC from Interface Options	42
Figure 6.19	Enabling the VNC Server	42
Figure 6.20	Creation and activation of the Python virtual environment	43
Figure 6.21	Upgrading pip in the Python Virtual Environment	43
Figure 6.22	Installation of PyTorch and TorchVision on Raspberry Pi	44
Figure 6.23	Installation of Required Python Libraries	44
Figure 6.24	Verification of Installed Python Libraries	45
Figure 6.25	Python Version Verification	45
Figure 6.26	Verification of PyTorch, TorchVision, and CUDA Availability	46

Figure 7.1	VisDrone Dataset Download Page on Kaggle	47
Figure 7.2	VisDrone Dataset Working Directory	48
Figure 7.3	Simplified Architecture of the Ultralytics YOLO11n Model Used for Object Detection	49
Figure 7.4	shows the training progress during the initial 20-epoch YOLO11n training process.	51
Figure 7.5	presents the command and initialization of the YOLO11n fine-tuning experiment.	53
Figure 7.6	Figure 7.6: Completion of YOLO11n fine-tuning and validation results of the final model	55
Figure 7.7	Transfer of the fine-tuned YOLO11n model to Raspberry Pi using SCP Figure	56
Figure 7.8	Verification of YOLO11n model files on Raspberry Pi	56
Figure 7.9	YOLO11n inference performance on Raspberry Pi CPU	57
Figure 8.1	Precision–Recall Curve of the Fine-Tuned YOLO11n Model	58
Figure 8.2	Confusion Matrix of the Fine-Tuned YOLO11n Model	60
Figure 8.3	Training and Validation Curves	61
Figure 8.4	Object detection results using fine-tuned YOLO11n	62
Figure 8.5	YOLO11n Object Detection Result on a VisDrone Aerial Image	62
Figure 8.6	YOLO11n Object Detection Result in a Low-Light VisDrone Scene	63
Figure 9.1	Raspberry Pi Camera Module V3 Preview and Verification	66
Figure 9.2	Verification of the Fine-Tuned YOLO11n Model on Raspberry Pi	67
Figure 9.3	YOLO11n Inference Pipeline Execution on Raspberry Pi	68
Figure 9.4	YOLO11n Object Detection Result on Raspberry Pi	69
Figure 9.5	Live YOLO11n Object Detection Using Raspberry Pi Camera Module V3	70
Figure 9.6	YOLO11n Inference Performance Metrics on Raspberry Pi CPU	71
Figure 10.1	Integrated Drone-Based Surveillance System	73
Figure 10.2	Integrated Hardware Setup of the Drone-Based Surveillance System	75
Figure 10.3	End-to-End Demonstration of YOLO11n Object Detection Using Raspberry Pi Camera Module V3	76
Figure 12.1	Proposed Autonomous UAV Mission and AI-Based Surveillance Flow	82
Figure 12.2	Proposed Autonomous Surveillance Mission Workflow	84

LIST OF TABLES

Table No.	Title of the Table	Page No.
Table 2.1	Hardware Requirements of the Proposed Drone-Based Surveillance System	7
Table 2.2	Software Requirements and Development Environment	8
Table 4.1	Final Hardware Configuration of the Proposed Surveillance System	25
Table 7.1	Training Environment and Experimental Configuration	50
Table 7.2	Configuration Parameters Used for YOLO11n Fine-Tuning	52–53
Table 7.3	Validation Performance Metrics of the Fine-Tuned YOLO11n Model	54
Table 8.1	Overall Evaluation Metrics of the Fine-Tuned YOLO11n Model	59
Table C.4	Deployed Project Files and Approximate Sizes	91
Table D.1	YOLO11n Model Training Configuration	93
Table D.2	Final-Epoch Training Losses from Initial YOLO11n Training	94
Table D.3	YOLO11n Fine-Tuning Configuration and Training Details	94
Table D.4	Dataset Scanning Results for the YOLO11n Training and Validation Dataset	95
Table D.5	Final-Epoch Training Losses from YOLO11n Fine-Tuning	95
Table D.6	Validation Performance of the Fine-Tuned YOLO11n Model	95
Table E.1	Representative Bus Image Detection Results	97
Table F.1	Drone Parts List	97
Table F.2	Embedded Vision System	97–98

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
BLDC	Brushless Direct Current
CSI	Camera Serial Interface
CPU	Central Processing Unit
ESC	Electronic Speed Controller
FPS	Frames Per Second
GPS	Global Positioning System
GPU	Graphics Processing Unit
IoT	Internet of Things
OS	Operating System
Pi	Raspberry Pi
PWM	Pulse Width Modulation
SSH	Secure Shell
UAV	Unmanned Aerial Vehicle
VNC	Virtual Network Computing
YOLO	You Only Look Once

CHAPTER 1

INTRODUCTION

1.1 Background

Drones, also known as **Unmanned Aerial Vehicles UAVs** are increasingly used for surveying, monitoring and surveillance. They are useful for observing hazardous areas because they can carry cameras and provide an aerial view that is difficult to obtain from the ground.

With embedded computing drones can do than just capture and transmit images. An onboard computer can process the captured images near the point where they are collected. This reduces the need to send all the data to a computer and can support more responsive surveillance applications.

This project, titled **"Intelligent Electronic Surveillance with Camera-Integrated Drones "** focuses on combining a drone platform, camera, embedded computing and AI-based object detection. The setup includes an **S550 hexacopter frame, Pixhawk flight controller, Raspberry Pi 4 and Raspberry Pi Camera Module V3**. The drone provides the platform the camera captures the images and the Raspberry Pi 4 is used for image processing and AI inference.

For object detection **YOLO11n** was. Fine-tuned using aerial images, including a portion of the VisDrone dataset. After training the model was transferred to the Raspberry Pi for inference. The required software environment was also configured with **PyTorch, TorchVision, OpenCV, NumPy and the Ultralytics YOLO framework.**

The Raspberry Pi was configured with Raspberry Pi OS, along with SSH and VNC for access. The trained model files were transferred using Secure Copy Protocol SCP and the required packages and model files were verified. The YOLO11n model was then loaded on the Raspberry Pi. Tested with camera input for object detection. The system displayed detected objects along with their confidence values and CPU-based inference was also checked to understand the performance of deep-learning-based detection, on the edge device.

Overall the internship brought together **drone hardware, flight control, embedded computing, camera-based image capture, computer vision, deep learning and edge AI** to develop a system of processing visual data onboard a drone.

1.2 Motivation

I was motivated to explore how a drone, a camera system and artificial intelligence can work together for surveillance. A drone can give a view of a place but the images become more useful when the system can also identify and analyze objects by itself.

During **my internship** I wanted to see how such a system could be built with easily available hardware. Using a Raspberry Pi 4 and a Raspberry Pi Camera Module V3 let me work with image processing and AI inference on a device. Using a Raspberry Pi 4 and a Raspberry Pi Camera Module V3 also helped me see the limits of running deep-learning models on a device versus a desktop computer.

Another motivation was to gain hands-on experience with the workflow instead of dealing with each technology separately. This meant working with the drone hardware setting up the Raspberry Pi capturing camera data, training and fine-tuning the YOLO11n model moving the trained model to the Raspberry Pi and testing object detection on the system.

Through this project I aimed to understand how drones, embedded systems, computer vision and deep learning can be combined to build a surveillance system. The work also helped me see the challenges of moving an AI model from a training environment to an edge device and gave me a basis for future work, in aerial surveillance and autonomous drone applications.

1.3 Role of Drones in Surveillance

Drones are useful for surveillance because they can carry cameras and other electronic gear to places that may be hard or dangerous for people to reach. From an altitude a drone can take pictures and cover a larger area than cameras on the ground giving a different view. Because of this drones are used for security monitoring, infrastructure inspection and search and rescue missions.

In this project the drone served not as an **Unmanned Aerial Vehicle (UAV)** but also as the main platform of the integrated surveillance system. The S550 hexacopter was used to carry the Raspberry Pi 4 and Raspberry Pi Camera Module V3. The camera captured the data, which was then processed by the onboard computing system using the **YOLO11n object detection model**. The combination of the drone, camera, embedded computer and AI model formed the components of the integrated surveillance system.

Therefore the role of the drone in this project went beyond just flying and moving. It acted as a platform for taking aerial pictures and became an important part of the visual surveillance workflow. The combination of the aircraft, onboard camera and AI-based processing unit gives a foundation, for development of more capable surveillance systems.

1.4 Role of Artificial Intelligence

Artificial Intelligence is a part of a surveillance system because it allows images captured by a drone to be analyzed automatically. Of relying only on manual observation Artificial Intelligence based computer vision can identify and recognize objects in images. This makes it easier to process and obtain information from a large collection of captured images.

In this project object detection was implemented using **YOLO11n**, a deep learning model. The model was. Fine-tuned using aerial image data and was later deployed on a **Raspberry Pi 4** for inference. The camera feed was processed using the computer vision system, which detected objects and marked their locations with bounding boxes along with confidence scores.

The integration of Artificial Intelligence therefore adds a processing layer to the drone based surveillance platform. Combining aerial image capture with Artificial Intelligence based object detection provides a basis for developing monitoring systems that can reduce dependence, on manual observation.

1.5 Lightweight Deep Learning and Edge Computing

Since a drone-based surveillance system is a small computing platform, it has very little memory and processing power when compared with that of a regular desktop PC or cloud server. Hence, the adoption of lightweight deep learning models is the way to go if deployment at an embedded level is the target. These small, efficient models are the way to go if we want high detection performance but also very manageable computational requirements.

We chose YOLO11n as the object detection model for its reasonably compact architecture, which makes it appropriate to be run on small computing devices. The model was first trained on a desktop computer followed by some parameter tuning, then it was deployed to the Raspberry Pi 4 to carry out experiments and run the model. It turned out that the model loaded nicely on the Raspberry Pi and object detection was done quite accurately and quickly.

The concept of edge computing means that image processing and object detection tasks are performed directly on Raspberry Pi rather than relying on centralized remote server or requiring a constant internet connection. This technique could lower data transmission needs and is a good first step towards a fully independent surveillance system. This study aimed to investigate the feasibility of deploying AI object detection model on embedded computing platform through Raspberry Pi-based deployment.

1.6 Project Scope

This project is about building and studying a drone-based surveillance system that can capture aerial images run on-board computing and detect objects using AI. We focus on assembling a Raspberry Pi 4 with a Pi Camera Module V3V3 and checking if the YOLO11n model can spot objects in pictures and videos.

The main stages of this project are: first we prepare data using the YOLO11n model with top-down images. Next we fine-tune the model's settings. Test the improved model. After training is finished we bring the model onto the Raspberry Pi. Run it in an edge-computing environment. The Raspberry Pi setup loads the model runs object detection draws bounding boxes and shows confidence scores.

We also. Integrate the main hardware of the flight platform. The parts are the S550 hexacopter body, Pixhawk flight controller, GPS, motors, ESCs and power system. We adjust flight-controller settings calibrate and carry out early flight tests as part of the system build.

Ultimately this work wants to see if an unmanned system with on-board AI vision can do surveillance without needing a server. This project does not cover automatic flight, advanced real-time optimization or fully autonomous surveillance missions because those topics are, for future study.

1.7 Report Organization

This report is structured into chapters that describe the activities performed during the course of the internship in a logical order from the general to the specific, beginning with background information and objectives and moving on to hardware design, AI-object detection, raspberry pi, and system-integration.

Chapter 1 - Introduction deals with the background information, objectives, motivations, drones and artificial intelligence, and scope of the project.

Chapter 2 - Project Objectives and Requirements highlights the objectives and requirements of the project, including functional requirements, hardware, and software requirements, and limitations.

Chapter 3 - System Architecture describes the system architecture of the proposed drone-based surveillance system and the interaction of the drones, on-board computer raspberry pi, camera, flight-control system, and the AI-based detection system.

Chapter 4 - Hardware Implementation deals with the hardware components and their integration in the proposed drone-based surveillance system.

Chapter 5 - Flight Controller Configuration and Drone Validation focuses on the configuration and calibration of the flight-controller and the initial test-flights and observations, as well as the lessons-learned during thevalidation process.

Chapter 6 - Software and Development Environment deals with the software used, the operating system, programming language, computer vision libraries, and AI-frameworks used during the development of the project.

Chapter 7 Dataset and Deep Learning Model deals with the creation of a dataset, YOLO11 n model, training, and fine-tuning to achieve good results in aerial object detection.

Chapter 8 Model Evaluation and Results highlight the results of the trained model using appropriate evaluation metrics and object detection results.

Chapter 9 Raspberry Pi Integration and Edge Inference focus on integrating the trained model into the raspberry pi, performing camera inference, and highlights the observations during edge-based object detection.

Chapter 10 Complete System Integration emphasizes the drone-based camera system, raspberry pi, and the overall AI-based object detection workflow that was deployed and achieved good results.

Chapter 11 Challenges and Engineering Observations discusses the challenges encountered while deploying the model and the drone, hardware-to-hardware connection, flight testing, and the learnings from the project.

Chapter 12 Future Scope emphasizes on the possible improvements and future developments like achieving autonomous drone-based real-time object detection, live video feed processing, and improved surveillance capabilities.

Chapter 13 Conclusion dwells upon the summary of the work performed during the internship, results achieved, and the overall outcomes of the project performed at the company.

Finally, the report consists of **References** and **Appendices** section consisting of relevant references, code snippets, commands, additional results, and relevant information associated with the project.

CHAPTER 2

PROJECT OBJECTIVES AND REQUIREMENTS

2.1 Project Objectives

The initial objective of the project was to research and develop a drone-based electronic surveillance system utilizing the onboard camera and AI-based processing. The work was focused on the development process of the drone that employed a Raspberry Pi-based image processing and YOLO11n-based object detection.

Thus, the project had the following objectives:

1. Research the structure and workings of the S550 hexacopter drone-based surveillance platform.
2. Prepare the platform for flight operations by studying the onboard Pixhawk flight controller, the GPS module, the Raspberry Pi 4 computer, and Pi Camera Module V3V3.
3. Research and prepare image data for the use in aerial surveillance and object detection.
4. Study and train the YOLO11n model of the object detection algorithm using aerial data.
5. Fine-tune the model and research detection performance using standard quality measures.
6. Transfer the trained model to the Raspberry Pi 4 computer for further research.
7. Research the possibilities of the object detection using the onboard camera and Pi-based environment.
8. Assess the possibilities and limitations of the AI-based object detection in the embedded platform for drone surveillance.
9. Conduct a brief configuration and test the drone setup for flight.

2.2 Functional Requirements

The functional requirements of the project were defined based on the intended operation of the drone-based surveillance and AI inference system. The system was expected to support the following functions:

1. **Aerial Image Acquisition:** The system should be capable of capturing images using the Pi Camera Module V3V3 mounted on the drone platform.
2. **Image Processing:** The captured images should be processed in a suitable format for AI-based object detection.

3. **Object Detection:** The trained YOLO11n model should identify relevant objects in the input images and provide bounding boxes with confidence scores.
4. **Embedded Inference:** The trained model should be transferred and executed on the Raspberry Pi 4 to study the feasibility of local, edge-based inference.
5. **Model and Camera Integration:** The Raspberry Pi should support the integration of the YOLO11n model with the camera-based detection pipeline.
6. **Drone Platform Operation:** The drone platform should support basic flight-controller configuration, calibration, and preliminary flight testing for aerial surveillance applications.
7. **Remote System Access:** The Raspberry Pi should support remote access through SSH and VNC/Raspberry Pi Connect for system configuration, file transfer, and software development.
8. **Result Visualization:** The detection output should display the detected object, bounding box, and corresponding confidence score for visual verification of the model's performance.

These requirements were used as the practical basis for developing and testing the individual components of the surveillance system during the internship.

2.3 Hardware Requirements

Table 2.1: Hardware Requirements of the Proposed Drone-Based Surveillance System

S. No.	Hardware Component	Specification / Role
1	S550 Hexacopter Frame	Aerial platform
2	Pixhawk 2.4.8	Flight controller
3	GPS Module	Positioning and navigation support
4	BLDC Motors	Drone propulsion
5	ESCs	Motor speed control
6	Radio Transmitter and Receiver	Manual flight control
7	Raspberry Pi 4 Model B	Embedded computing and AI inference
8	Raspberry Pi Camera Module V3	Image acquisition
9	3S LiPo Battery (5200 mAh)	Drone power supply
10	Power Bank (10000 mAh)	Raspberry Pi power during testing
11	MicroSD Card	Raspberry Pi OS and system storage

2.4 Software Requirements

The software environment was chosen for the configuration of the drone, image processing, deep learning model design, development, and deployment on the Raspberry Pi platform. The list of the main software tools and frameworks used for the development includes:

Table 2.2: Software Requirements and Development Environment

S. No.	Software / Framework	Purpose
1	Raspberry Pi OS (64-bit)	Operating system for the Raspberry Pi 4
2	Python 3.13.5	Programming and development environment
3	PyTorch 2.6.0+debian	Deep learning framework for model inference
4	TorchVision 0.21.0	Computer vision utilities supporting PyTorch
5	OpenCV 5.0.0	Image and camera processing
6	NumPy 2.2.4	Numerical and array-based data processing
7	Ultralytics YOLO 8.4.102	YOLO11n model implementation and inference
8	Raspberry Pi Imager v2.0.10	Raspberry Pi OS installation and system configuration
9	Mission Planner	Drone flight controller configuration and calibration
10	SSH / SCP	Remote terminal access and model/file transfer
11	VNC / Raspberry Pi Connect	Remote graphical access to the Raspberry Pi

2.5 Scope and Limitations

The scope of this project includes the development and evaluation of a drone-based electronic surveillance system that incorporates aerial imaging, onboard computing, and artificial intelligence (AI)-assisted object detection. The project involves the research and integration of unmanned aerial vehicle (UAV) components such as the S550 hexacopter, Pixhawk flight controller, Raspberry Pi 4, Pi Camera Module V3V3, and a lightweight YOLO11n object detection model.

The project further includes the preparation of the deep learning model for execution, deployment on the Raspberry Pi computer, and testing of the model's ability to perform object detection as an edge-computing device. The initial configuration and calibration of the drone and its flight-testing were also conducted to establish the performance and practicality of the proposed system.

The current project has several constraints. The inference on the Raspberry Pi platform was done using the CPU which limited the processing speed and increased the time taken to process

individual images. The project's scope only included the initial steps in the end-to-end surveillance system pipeline, leaving out a fully autonomous surveillance mission. Further, the project did not involve continuous video inference from the aerial drone, optimizing the deep learning model for the edge computer, or conduct extended field testing. These constraints define the limitations of the current project and suggest areas for future improvements.

CHAPTER 3

SYSTEM ARCHITECTURE

3.1 Overall System Architecture

The overall system architecture can be broadly divided into two primary segments: the drone platform required to perform aerial surveillance and the embedded computer vision and AI system needed to perform image acquisition and object detection.

The drone platform is constructed using an S550 hexacopter frame which houses the Pixhawk 2.4.8 flight controller, GPS, brushless motors, ESCs, and radio control system. The flight controller is responsible for the basic piloting and stability of the drone during flight.

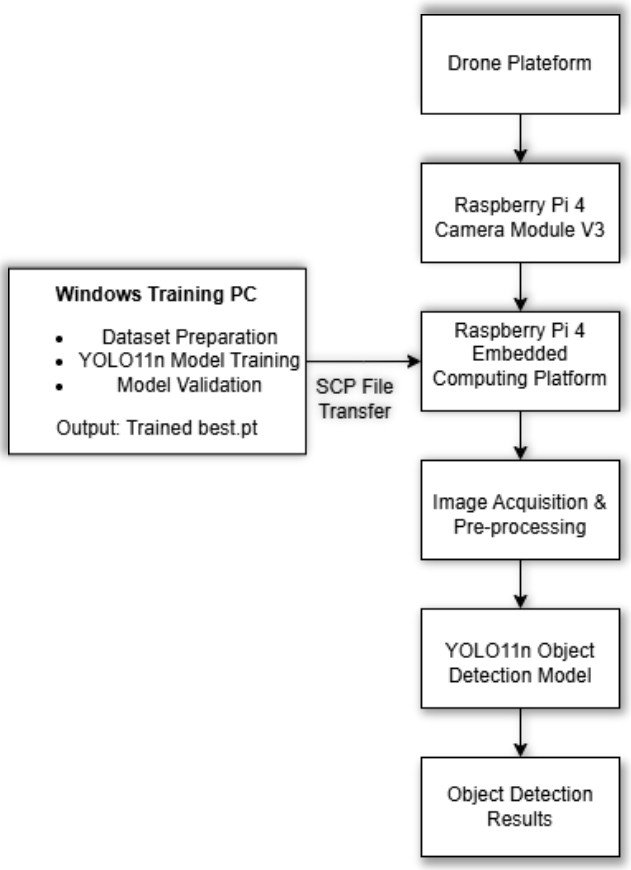


Figure 3.1: Overall System Architecture of the Drone-Based Electronic Surveillance System

The vision and AI system is composed of the Raspberry Pi 4 computer and Pi Camera Module V3V3 stereo camera. The camera is used to capture imagery, and the computer serves as the embedded processor for performing the image processing and object detection. The YOLO11n

algorithm is used for object detection, with the model being run on the Raspberry Pi to produce an image with bounding boxes and confidence scores for detected objects.

The model itself is trained first in the training environment before being transferred to the raspberry pi for inference. With that, the following architecture can be proposed:

The system architecture is overall focused on the concept of aerial surveillance where the drone is used to capture images from above, and with the vision and AI system process them in order to detect and identify objects.

3.2 Major Subsystems

The developed surveillance system comprises two primary subsystems that ensure the intelligent flight of the drone and conduct image analysis and object detection. The first subsystem allows the drone to perform stable flights, navigate, and maintain control. The second subsystem is responsible for obtaining images from the drone's camera and conducting computer vision tasks with the use of an artificial intelligence algorithm. By integrating these two components, the drone can acquire images from above and process it with the embedded computer vision model.

The two identified subsystems include the following:

- Flight Control Subsystem
- Embedded Vision Subsystem

3.2.1 Flight Control Subsystem

The Flight Control subsystem manages the stabilization, control, and navigation of the drone during the flight according to the pilot's commands sent from the radio control transmitter. This system reads the information from the sensors and regulates the motors' speed to maintain the stable movement of the drone. In this subsection, I will discuss the details about the selected flight controller, the integration of this component with the rest of the drone, and the performed tests to ensure that the controller is working correctly.

The Flight Control subsystem was built based on the Pixhawk 2.4.8 flight controller. The controller communicates with the GPS module, ESCs, motors, and radio receiver, providing the closed-loop control system. The following photo shows the drone's radio control receiver, GPS module, ESCs, and brushless DC motors integrated into the Flight Control subsystem. All calibration procedures for accelerometers, compass, radio control, and ESCs were completed using Mission Planner before performing the test flights.

The Flight Control subsystem has several key functions, including:

- Flight stabilization
- Motor speed control
- Sensor data processing
- GPS-based positioning
- Radio command processing
- Safe flight operation



Figure 3.2 – Complete Drone Platform Used for the Surveillance System



Figure 3.3 – Radio Transmitter Used for Manual Flight Control

3.2.2 Embedded Vision Subsystem

The Embedded Vision Subsystem offers image capture and onboard object detection by implementing artificial intelligence. This subsystem consists of a Raspberry Pi 4, Pi Camera Module V3 Module V3, and utilizes the lightweight YOLO11n deep learning model to facilitate the on-device image detection of surveillance images.

The Raspberry Pi is used as a local edge computational device on the embedded vision subsystem, while the Pi Camera Module V3V3 captured the images under surveillance. These captured images were then processed using the YOLO11n model, which was trained to detect objects of interest from the surveillance images locally rather than relying on cloud-based processing. This way, the embedded system has exhibited capability to perform intelligent visual surveillance on an edge device with limited computational resources.

The following are the primary functions of the Embedded Vision Subsystem:

- Image acquisition
- Image preprocessing
- Object detection
- Image classification
- Edge AI inference
- Display and storage of detection results



Figure 3.4 – Raspberry Pi 4 and Pi Camera Module V3 Module V3 Mounted on the Drone Platform



Figure 3.5 – Raspberry Pi Camera Module V3 Used for Image Acquisition

3.3 System Block Diagram

The overall system workflow of the developed surveillance system is presented in Figure 3.3. The drone is used to capture images from the air, while the Raspberry Pi 4 is the local edge server that processes these images. Specifically, images captured with the Raspberry Pi Camera Module V3 are processed using the object detection model, i.e., YOLO11n weights, and the results are displayed and saved for further analysis.

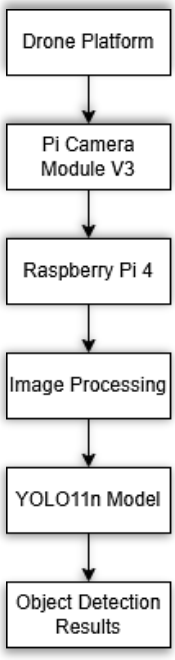


Figure 3.6: System Block Diagram of the Intelligent Electronic Surveillance System

3.4 Power Architecture

The developed surveillance system consists of two power sub-systems: the drone platform and the embedded vision unit. Two separate power supplies were used for the drone platform and the embedded computer due to their differing power needs.

The **3S LiPo battery (5200 mAh)** is used to power the flight controller, ESCs, BLDC Motors, GPS module, and radio receiver of the drone during flight operations. The battery above is used to provide power to the propellers of the droneas well as the flight control computer ensuring stable and safe flight.

The embedded vision system, which consists of the **Raspberry Pi 4** and the **Raspberry Pi Camera Module V3** is powered by a separate **10000 mAh power bank.** Using a separate power source ensures that the 5 V power supply to the Raspberry Pi remains stable and uninterrupted, and it also allows the Raspberry Pi to be powered without having to turn on the drone motor. Using a separate power source improves system stability and safety and reduces the overall complexity of the system. Furthermore, the separate power source allows testing of the raspberry pi computer, embedded computer vision models, software, and image capture capabilities independently from the flight computer prior to integration and deployment.



Figure 3.7: 3S LiPo Battery (5200 mAh) Used for Powering the Drone Platform



Figure 3.8: 10,000 mAh Power Bank Used for Supplying Power to the Raspberry Pi 4

3.5 Communication and Control Architecture

The communication and control architecture of the developed surveillance system was established by integrating the flight control subsystem and the embedded vision subsystem. The two systems operate independently but in conjunction with each other to achieve aerial imaging and intelligent object detection.

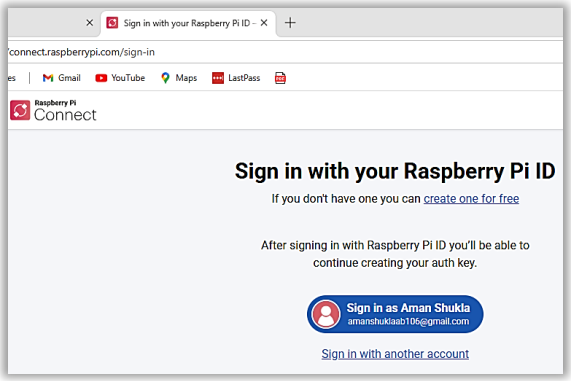
The **Pixhawk 2.4.8 flight controller** serves as the central unit responsible for maintaining the stability and control of the drone. Flight commands are transmitted through the **FlySky FS-i6 radio transmitter** and received by the onboard receiver, allowing the pilot to manually control the drone during testing and validation. Mission Planner software was used for configuring the flight controller, calibrating sensors, monitoring system parameters, and verifying the overall readiness of the drone before flight operations.

The camera surveillance system is run by the Raspberry Pi 4 processor, to which the camera module is connected via the CSI interface. The video feed from the camera is processed by the Pi using the YOLO11n model, which has been trained to perform object detection. In this way, the embedded vision system is capable of running an artificial intelligence model without dependence on the cloud.

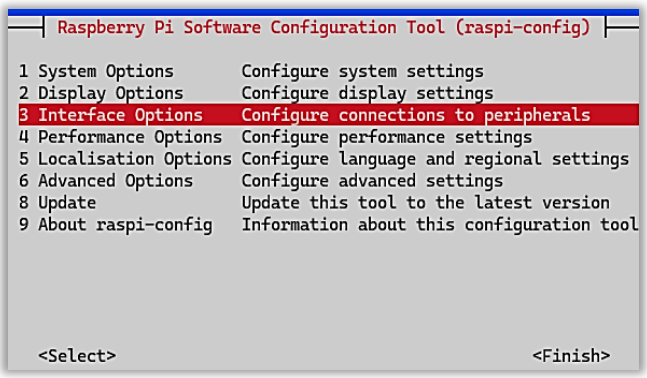
Several remote access services were configured on the Pi to enable remote access to the system. These services included Secure Shell (SSH) for remote terminal access, Visual Network Computing (VNC) for desktop access, and Raspberry Pi Connect for remote Raspberry Pi management. In this way, remote management of the surveillance system became possible, thereby facilitating the development process.

Overall, the communication and control architecture of the developed drone surveillance system is flexible, allowing the flight and vision systems to operate independently. Specifically, the former subsystem ensures stable flight, while the latter performs image processing and object detection. In addition, the architecture is advantageous in terms of facilitating the upgrading process of both systems.

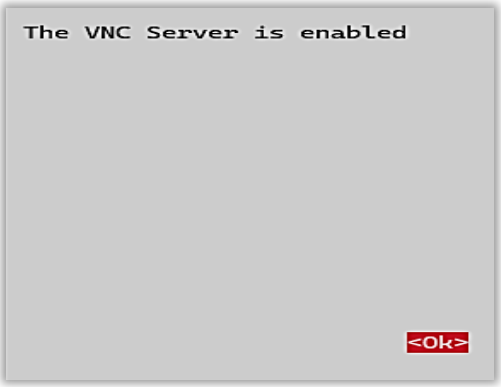
During the system integration phase, the remote access to the Raspberry Pi was set up for the convenience of software development, model deployment, and system observation. For this purpose, Raspberry Pi Connect was authorized with a Raspberry Pi ID, VNC Server was installed through the raspi-config menu, and the Raspberry Pi IP address was identified using the hostname -I command, thus establishing a remote access procedure to the embedded system throughout the project's development.



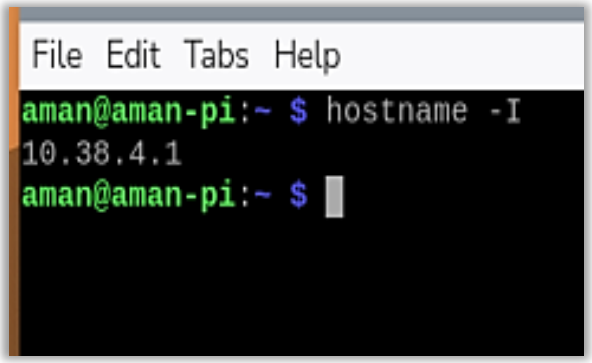
(a) Raspberry Pi Connect Authentication



(b) VNC Configuration through raspi-config



(c) VNC Server Successfully Enabled



(d) Raspberry Pi IP Address Verification

Figure 3.: Configuration of Remote Access Services on the Raspberry Pi

The Raspberry Pi was configured for remote access using Raspberry Pi Connect and VNC Server, with its IP obtained via hostname -I.

CHAPTER 4

HARDWARE IMPLEMENTATION

4.1 S550 Hexacopter Platform

The surveillance system was designed using an **S550 hexacopter frame** which was used to mount the flight controller, computer, camera, and power system. The frame provides a good balance between lifting capability and stability during flight.

The hexacopter has a total of six brushless DC motors mounted on the body to provide lift and stability during flight. In addition, the model's modularity allowed the installation of the Pixhawk flight controller, GPS, Raspberry Pi 4, Pi Camera Module V3 Module V3, battery, and other components used to build the prototype.

The S550 frame was chosen for its lightness, simplicity, and compatibility with Pixhawk flight controllers. All these factors combined made it an ideal candidate for prototyping the intelligent electronic surveillance system under consideration.



Figure 4.1: S550 Hexacopter Platform Used for the Development of the Surveillance System

4.2 Pixhawk Flight Controller

The **Pixhawk 2.4.8 flight controller** was the main onboard computer responsible for receiving input from the radio transmitter, processing information from sensors, and controlling the speed of the Electronic Speed Controllers (ESC).

The flight controller was programmed using the **Mission Planner software** during the internship. The following settings were configured in the “Initial Setup”: firmware, frame, accelerometers, compass, radio, ESCs, flight modes, and failsafe. As a result, appropriate communication with onboard equipment was established, which allowed for achieving stable flight during test sessions.

In particular, I established links between the flight controller and the GPS module, radio receiver, ESCs, and power module. The Pixhawk 2.4.8 flight controller was used for preliminary flight tests of the drone before installing the embedded computer.



Figure 4.2: Pixhawk 2.4.8 Flight Controller Installed on the Surveillance Drone

4.3 Brushless Motors and ESCs

The propulsion system of the surveillance drone comprises of **six brushless direct current (BLDC) motors** and the associated **electronic speed controllers (ESC).** The BLDC provides the necessary thrust for taking off, hovering, and maneuvering the drone. The ESCs control the speed of the BLDC motors based on the signal received from the PixHawk flight controller.

I synced each ESC to the PixHawk and calibrated it before testing the motors. The process is essential to ensure that each motor delivers the same level of response so that it can generate uniform thrust. Additionally, the calibration process ensured that each ESC responded effectively to the speed modulation signal from the flight controller without any lag.



Figure 4.3: Brushless DC Motors Installed on the S550 Hexacopter Platform

4.4 GPS Module

The surveillance drone used a **u-blox NEO-M8N GPS module** with a built-in digital magnetometer (electronic compass). The GPS module was connected to the **flight controller** to provide positioning information while in flight.

The GPS module was connected to the Pixhawk **GPS** and **I2C Compass** ports, and it was configured and tested in the **Mission Planner software** during the setup stage. Satellite reception and compass detection were achieved, and the sensors were calibrated before test flights began.

In order to avoid interference from the ESCs, motor wiring, and power distribution, the GPS was mounted on a high place on a mast. The GPS module was also mounted facing forward in the direction of flight so that the flight controller could calculate the proper heading.

The GPS and compass system provided the navigation information while performing the initial flight test as part of the internship.



Figure 4.4: u-blox NEO-M8N GPS Module with Integrated Digital Compass Mounted on the Drone

4.5 Radio Transmitter and Receiver

A **Fly Sky FS-i6 Digital Radio Transmitter** was used to manually control the surveillance drone during configuration and flight testing. The radio utilizes the **2.4 GHz AFHDS 2A protocol**; it communicates with the **Fly Sky FS-iA6B** receiver, providing stable radio communication with the Pix hawk 2.4.8 flight controller.

The receiver was connected to the **RC IN** port of the flight controller using the **PPM signal (summed channel outputs)**. The radio control was calibrated in the Mission Planner software by setting up the throttle channel, roll, pitch, yaw, and auxiliary channels.

Prior to the flight tests, the transmitter and receiver links were checked, the battery voltages were measured, and radio control channels were tested.



Figure 4.5: Fly Sky FS-i6 Radio Transmitter Used for Manual Drone Control

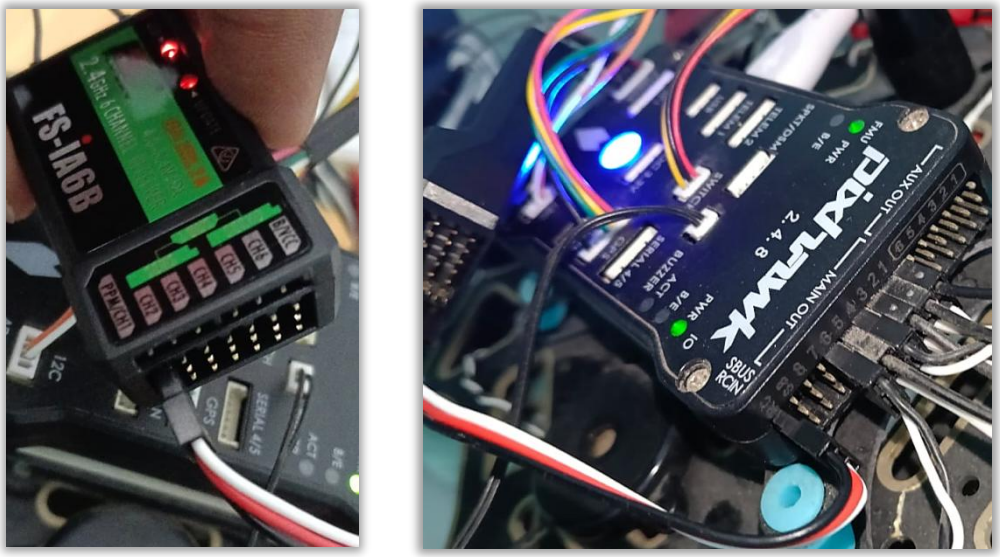


Figure 4.6: Fly Sky FS-iA6B Receiver Connected to the Pixhawk Flight Controller

4.6 Raspberry Pi 4

The **Raspberry Pi 4 Model B (4 GB)** was utilized as an onboard edge computing device for the proposed surveillance system. It provided model inference using the Python application while interacting with the Raspberry Pi Camera Module V3 and performing onboard image processing using the YOLO11n algorithm.

The **Raspberry Pi OS (64-bit), i.e., Debian GNU/Linux 13 (Trixie)**, was installed on the edge computing device. Furthermore, **Python 3, OpenCV**, and **Ultralytics packages** were installed in the system to enable deep learning-based model inference. The **Secure Shell (SSH), Raspberry Pi Connect,** and **Virtual Network Computing (VNC)** were activated to enable remote access to the system for software installation and configuration.

The surveillance system was configured using the installed Python application and **YOLO11n** algorithm to perform onboard inference of captured images. The results demonstrated the ability of the surveillance system to perform intelligent electronic surveillance using artificial intelligence and deep learning-based algorithms running on a low-cost, embedded computing device, i.e., Raspberry Pi 4 Model B.



Figure 4.7: Raspberry Pi 4 Model B (4 GB RAM) Used in the Proposed Surveillance System

4.7 Raspberry Pi Camera Module V3

The **Raspberry Pi Camera Module V3** was employed as the primary imaging hardware component. It was connected to the camera interface of the Raspberry Pi 4 by using the **CSI**

(Camera Serial Interface) ribbon cable. It is possible to capture images directly from the Camera Module for further processing by the computer vision system on the Raspberry Pi board.

The sensor of the Camera Module 3 is the **Sony IMX708 CMOS image sensor** with a **12 MP** resolution. It allows to obtain images of sufficient quality for performing computer vision tasks. In particular, during the internship, surveillance images were captured using this camera, which were then processed by the **YOLO11n model** for detecting and classifying objects.

The Raspberry Pi Camera Module V3 was a reasonable choice due to its compact size and weight. Besides, it can be directly integrated into the Raspberry Pi platform without any additional components, which makes it convenient for use in the embedded vision system of the drone.

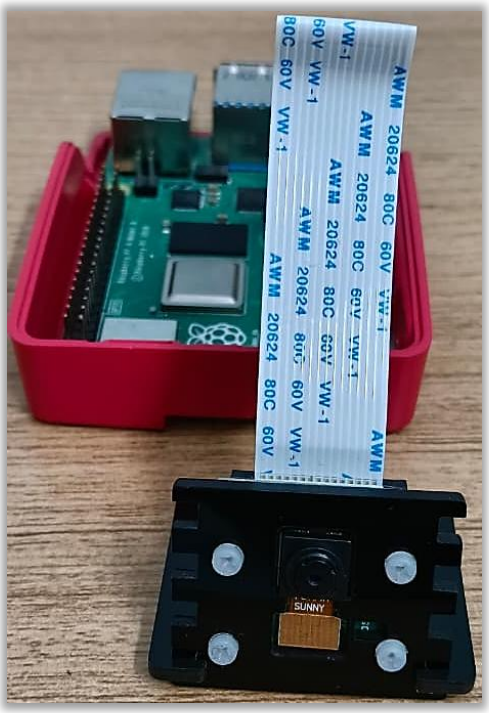


Figure 4.8: Raspberry Pi Camera Module V3 Connected to Raspberry Pi 4 via CSI Interface

4.8 Payload and Weight Estimation

The payload of the S550 hexacopter was estimated from the selected propulsion system and the additional surveillance electronics. The drone uses six 2212 920KV brushless motors, 1045 (10×4.5) propellers, 30 A ESCs, and a 5200 mAh 3S LiPo battery.

Each 2212 920KV motor with the selected 1045 propeller configuration is considered to provide approximately 500 g maximum static thrust.

Total Theoretical Thrust

Total Thrust = Number of Motors × Thrust per Motor

= 6 × 500 g = 3000 g ≈ 3.0 kgf

6 motors × 500 g ≈ 3.0 kg maximum total thrust

The additional surveillance payload consists of the Raspberry Pi 4 (≈47 g), Raspberry Pi Camera Module V3 (≈4 g) and the specified power bank (≈280 g).

Total Surveillance Payload

Payload = Pi 4 + Camera V3 + Power Bank

= 47 g + 4 g + 280 g

= 331 g

Pi 4 + Camera V3 + Power Bank ≈ 331 g

Thus, the complete additional surveillance payload considered in the present configuration is approximately 331 g. The battery, motors, ESCs, Pixhawk, GPS, frame and other flight-control components are considered part of the drone's own all-up weight and are therefore not included in this payload value.

The payload capacity of the complete drone can be determined after measuring the final assembled drone weight:

Payload Capacity

Maximum Payload = Allowable AUW − Existing Drone Weight

The calculated 3.0 kg represents the theoretical maximum propulsion thrust and should not be interpreted as a 3.0 kg payload capacity. Actual safe payload capacity depends on the measured all-up weight, available thrust margin, battery condition and flight-test performance.

4.9 Final Hardware Configuration

After the assembly and configuration of all hardware components, the surveillance platform was completed as an integrated system. The assembled configuration included the S550 hexacopter frame, Pixhawk 2.4.8 flight controller, u-blox M8N GPS module with a compass, FlySky FS-i6

radio controller, brushless motors with ESCs, Raspberry Pi 4 Model B, and Raspberry Pi Camera Module V3. All components were properly mounted and connected to each other to enable the system's testing and validation.

The Raspberry Pi processor and camera were at the center of the embedded vision system. In addition, the flight controller was responsible for the drone's flight management and stabilization. The described hardware configuration enabled me to perform image acquisition, conduct object detection using artificial intelligence algorithms, and complete basic flight tests during my internship.

Table 4.1: Final Hardware Configuration of the Proposed Surveillance System

Component	Model / Specification	Purpose
Drone Frame	S550 Hexacopter	Aerial platform
Flight Controller	Pixhawk 2.4.8	Flight control
GPS Module	u-blox M8N with Compass	Positioning and navigation
Radio Transmitter	Fly Sky FS-i6	Manual control
Radio Receiver	Fly Sky FS-iA6B	Signal reception
Embedded Computer	Raspberry Pi 4 Model B (4 GB)	Edge computing
Camera	Raspberry Pi Camera Module V3	Image acquisition
Battery	3S 5200 mAh LiPo	Drone power supply
External Power	Xiaomi 4i 10000 mAh Power Bank	Raspberry Pi power

CHAPTER 5

FLIGHT CONTROLLER CONFIGURATION AND DRONE VALIDATION

5.1 Pixhawk Configuration Using Mission Planner

The **Pixhawk 2.4.8 flight controller** was configured using **Mission Planner (Version 1.3.83)** as the Ground Control Station (GCS) software. The controller was connected to the computer through a USB interface and detected on **COM9** with a communication speed of **115200 baud**, where the connection was established as **COM9-1-HEXAROTOR**.

During the initial setup, the frame **Copter V4.6.3 OFFICIAL** firmware package was selected from the Mission Planner firmware installation menu. After the firmware installation, the **Hexa rotor (X)** frame type was configured to match the S550 drone platform used in this project. The firmware installation and board configuration were verified successfully before proceeding with the remaining setup procedures.

Once the firmware and frame configuration were completed, the Pixhawk flight controller was ready for sensor calibration, radio configuration, ESC calibration, flight mode assignment, and preliminary flight testing.



Figure 5.1: Mission Planner Connected to Pixhawk Flight Controller (COM9, 115200 Baud)

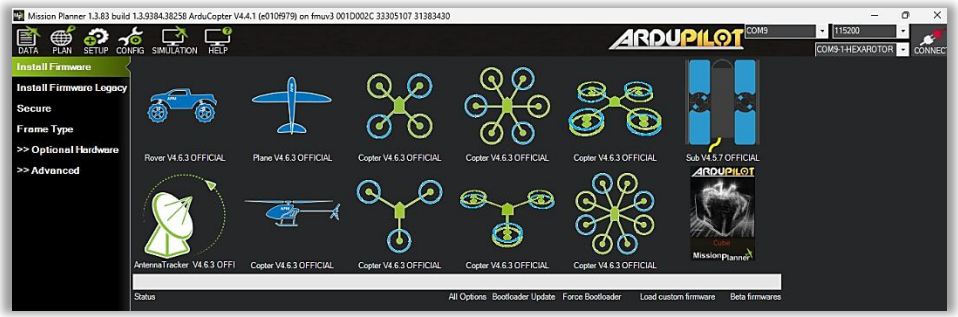


Figure 5.2: Hexa rotor (X) Frame Configuration in Mission Planner

5.2 Sensor Calibration

In order for the aircraft to have stable flight characteristics and for navigation to be accurate, it is necessary to carry out a calibration of all sensors. All calibration procedures were done properly before flight tests using Mission Planner.

5.2.1 Accelerometer Calibration

The accelerometer was calibrated using **Mission Planner** so that it could accurately estimate the attitude of the drone during flight. In order to do this, the calibration tool within Mission Planner (Setup → Mandatory Hardware → Accel Calibration) was used after connecting to the Pixhawk flight controller.

The drone was placed on a flat surface and placed in the six different orientations specified by Mission Planner: **level, left side, right side, nose down, nose up, and inverted**. It was held in each position until Mission Planner registered the change in orientation. Once all six positions had been successfully entered, Mission Planner notified the user that the **calibration was successful**, which meant that the offsets for the accelerometers had been saved.

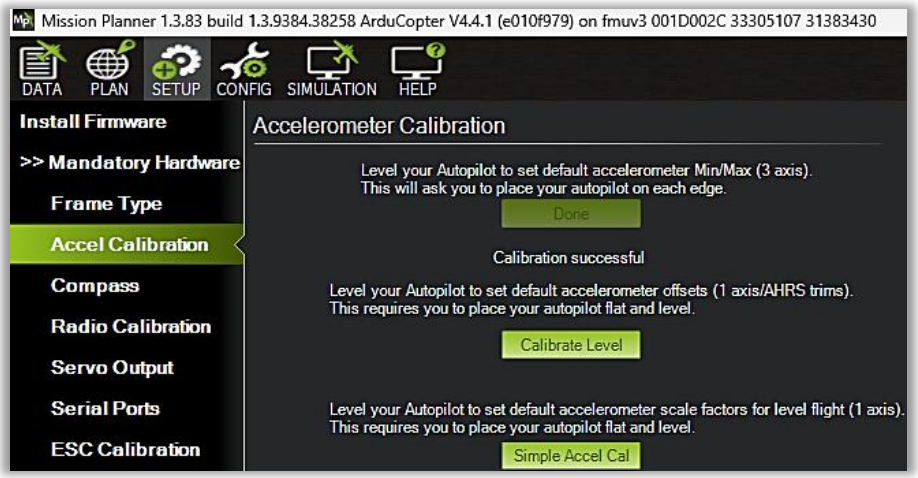


Figure 5.3: Accelerometer Calibration Completed Successfully

5.2.2 Compass Calibration

The calibration of the compass was performed by using Mission Planner to ensure that the heading estimation is accurate, and the drone flies properly. It was done by clicking on Setup, after that choose option **Mandatory Hardware**, in this choose the option **Compass** after connecting Pixhawk flight controller and Mission Planner.

Before calibration, an external u-blox M8N GPS + HMC5883 was set as the primary compass, and an LSM303D was set as the secondary one. The calibration was started by going to **Onboard Mag Calibration** option, then slowly rotating a drone on all axes to provide the necessary data to Mission Planner so that it could calculate the offsets. In the process of calibration, it was needed to wait while each progress bar reached 100% for onboard and external compasses. Then, Mission Planner provided magnetic offset values and showed **MAG_CAL_SUCCESS** for two compasses. Finally, the flight controller was rebooted to apply new settings.

After calibration, the heading estimation was accurate enough to allow the GPS-based flight modes, such as Loiter, Auto, and RTL, to work properly.

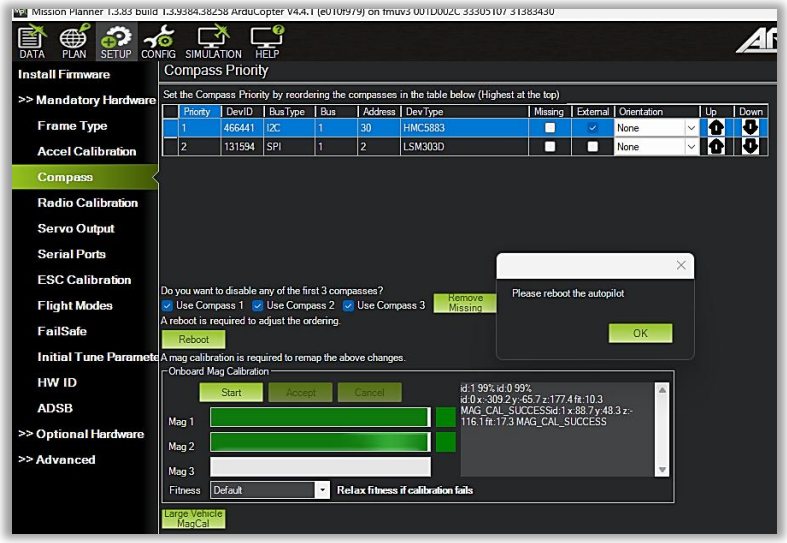


Figure 5.4: Compass Calibration in Mission Planner

5.2.3 Radio Calibration

Radio calibration was performed using Mission Planner software in order to establish proper communication between the **Fly Sky FS-i6 transmitter and FS-iA6B receiver** and Pixhawk 2.4.8 flight controller. Before proceeding to calibration, all propellers were removed and placed in the same position (CW/CCW) as they were before the calibration procedure in order to ensure safety and easy installation after the calibration is finished. The Pixhawk was connected to

the computer using the COM9 port with 115200 baud rates, and the **Radio Calibration** option was opened by navigating to Setup then click on the Mandatory Hardware, after that, the calibration process was initiated by clicking Calibrate Radio. During the calibration process, the transmitter sticks and switches were moved from the lowest to the highest position in order to let Pixhawk record min, max, and neutral values for each channel.

After calibration, the following channel ranges were observed in the Mission Planner:

- **CH1:** 1000-1999 µs (Roll)
- **CH2:** 999-2000 µs (Pitch)
- **CH3:** 999-1995 µs (Throttle)
- **CH4:** 999-1999 µs (Yaw)

which means that the calibration was successful, and the range of travel for each channel was saved by the flight controller, which allows for manual flight without calibration errors.

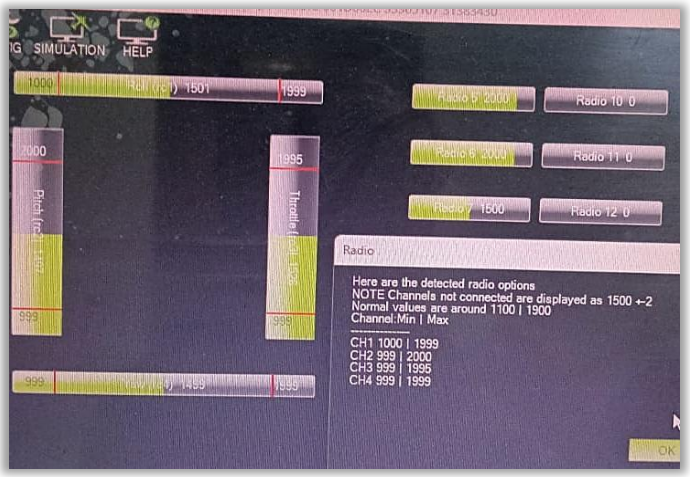


Figure 5.5: Successful Radio Calibration in Mission Planner

5.2.4 ESC Calibration

The Electronic Speed Controllers (ESC) were calibrated using the **Mission Planner software** in order to balance the throttle range of the **Pixhawk 2.4.8 flight** controller and the six ESCs. All the propellers were removed prior to calibration and left in their original positions (CW/CCW) for safety reasons.

The ESC Calibration tab under the Setup, click on the Mandatory Hardware section was opened and the calibration procedure described below was performed by following the standard Ardupilot procedure. The procedure ensured that the flight controller learned the maximum and minimum

throttle values sent from the Fly Sky FS-i6 radio and saved them to all the ESCs. After the procedure, all the motors reacted to control inputs in the same manner.

- **ESC Protocol:** DShot1200
- **Output PWM Range:** 1000-2000
- **Spin When Armed:** 0.100
- **Spin Minimum:** 0.150
- **Spin Maximum:** 0.950

The successful ESC calibration allowed to synchronize the behavior of all the motors and improve the flight performance during the test flights.

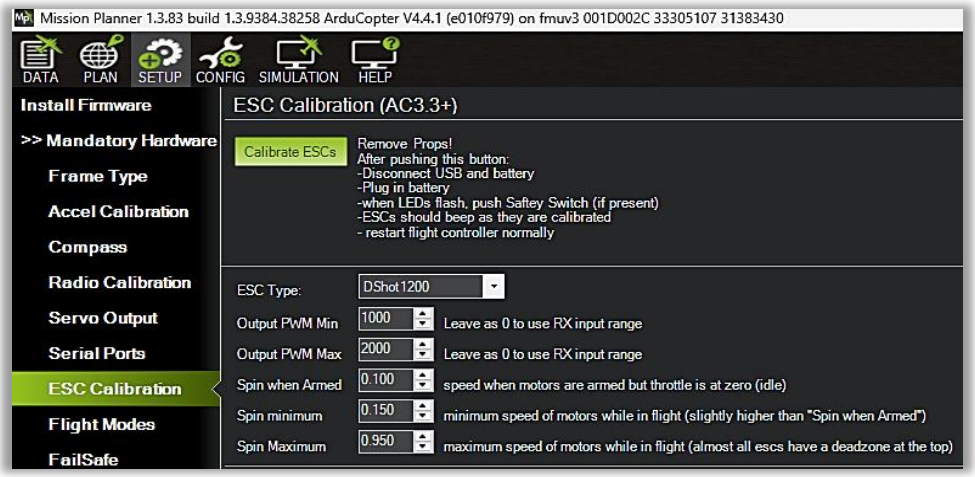


Figure 5.6: ESC Calibration Settings in Mission Planner

5.3 Flight Mode Configuration and Failsafe Settings

The flight modes and the failsafe parameters of the Pixhawk 2.4.8 flight controller were set using the Mission planner in order to establish the safe and reliable operation of the hexacopter.

The flight modes were set using Channel 5 of the radio transmitter:

- Flight Mode 1 – **Stabilize**
- Flight Mode 2 – **Alt Hold**
- Flight Mode 3 – **Loiter**
- Flight Mode 4 – **Auto**
- Flight Mode 5 – **RTL (Return-to-Launch)**
- Flight Mode 6 – **Land**

The set flight modes were saved by clicking **Save Modes** in the Mission Planner.

The failsafe parameters were set in the **Fail Safe** menu. The Radio Failsafe was set to Enabled Always RTL with FS PWM equal to 975 µs. The **3S (11.1 V/5200 mAh)** LiPo battery had the following set Low Battery Voltage of **10.8 V, Reserved** Capacity set to **1040 mAh**, and Low Battery Timer set to 10 s. The option GCS Failsafe was disabled.

The set failsafe parameters will allow a safe operation of the drone even if the radio control signal is lost by enabling Return-to-Launch (RTL). In addition, the set battery parameters will prevent the drone from operating at low battery levels.



Figure 5.7: Flight mode and failsafe configuration in Mission Planner.

5.4 Flight Testing

The flight tests were conducted in an open outdoor area after the completion of the hardware installation and adjustment. The hexacopter was first tested at a **low altitude (approximately 0.5-1m above the ground)** to ensure proper take-off, hover, and control. The drone was then flown at a **medium altitude (approximately 25-35 m) and then at approximately 35-50m** to check the stability of the aircraft during the flight at a normal operating altitude. The forward flight was stable, requiring only occasional attitude adjustments to counteract the wind effects.



Figure 5.8: Low-altitude hover during initial flight testing.



Figure 5.9: Medium-altitude flight above the surrounding area.



Figure 5.10: Higher-altitude flight demonstrating stable operation.

5.5 Preliminary Flight Validation

The flight tests validated that the Pixhawk 2.4.8 flight controller, GPS, ESCs, motors, and radio control functioned as expected. Smooth hover was achieved at low altitudes, while higher altitude tests maintained stable forward flight without significant wobble. The flight modes were all correctly set up and responded as expected, certifying the platform for the integration of the camera cluster, the Raspberry Pi 4 computer, and the lightweight YOLO11n model for object detection.

5.6 Observations and Lessons Learned

The test flights highlighted that correct calibration of the compass, radio, and ESCs was necessary for stable flight. Smooth low-altitude hover was a good way to validate that the controls were responsive enough for higher altitude flights. The drone was generally highly stable during the test flights, while flying in an open area allowed avoiding obstacles and providing the GPS with a clear view of satellites. The preliminary test flights certified the platform for AI-enabled aerial surveillance experiments.

CHAPTER 6

SOFTWARE AND DEVELOPMENT ENVIRONMENT

6.1 Software Environment

The software environment used in the project included such means as Mission Planner, Raspberry Pi OS, 64-bit Python, OpenCV, and Ultralytics YOLO. These were used for flight controller configuring, embedded system development, image processing, and deploying a lightweight YOLO11n object detection model on the Raspberry Pi 4 computer, respectively.

6.2 YOLO11n Model

YOLO11n (You Only Look Once 11 Nano) is a lightweight object detection model that was created as part of the Ultralytics YOLO algorithm collection. The model was chosen for this project due to its relatively low system requirements and ability to run on lower-end hardware such as the raspberry pi 4. The model can detect objects in images with a single pass, which allows for quick processing and is useful for the drone's camera. The model was trained on the VisDrone dataset and deployed on the raspberry pi for on-board processing.

6.3 Development Environment Setup

A working environment was deployed on the Raspberry Pi 4 for the implementation of the YOLO11n object detection model. The environment was prepared by installing the Raspberry Pi OS, setting up the network interface, creating a python virtual environment, installing the necessary deep learning packages, and verifying the working environment for model deployment.

The development process is described in detail below.

6.3.1 Raspberry Pi OS Installation and Configuration

The Raspberry Pi Imager software was opened to prepare the microSD card for the operating system installation. In the **Device** section, the **Raspberry Pi 4** model was selected, as it was the hardware platform used for this project.

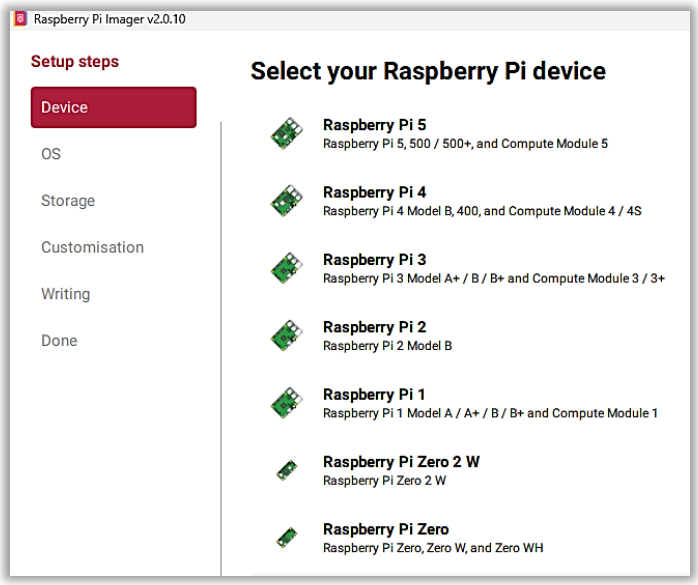


Figure 6.1: Selection of the Raspberry Pi 4 device in Raspberry Pi Imager.

Step 2: OS Selection

After selecting the device, it was necessary to choose the operating system in the Raspberry Pi Imager. In this case, the option **Raspberry Pi OS (64-bit)** was selected as the optimal solution for working with the Raspberry Pi 4 (4 GB) computer kit, since it includes the necessary deep learning libraries and software packages.

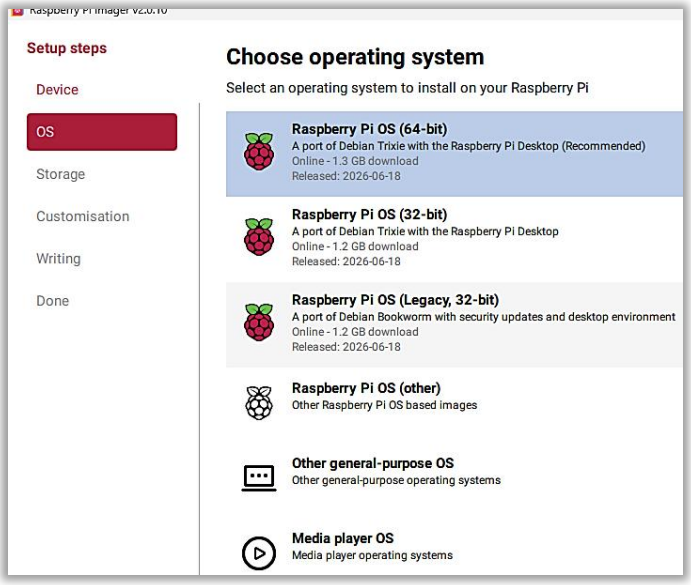


Figure 6.2: Selection of Raspberry Pi OS (64-bit) in Raspberry Pi Imager.

Step 3 – Storage Selection

After selecting the operating system, the storage device (microSD card) is selected where the system will be installed. The Exclude System Drives option is turned on in order to avoid selecting, installing or formatting the computer's built-in storage by mistake

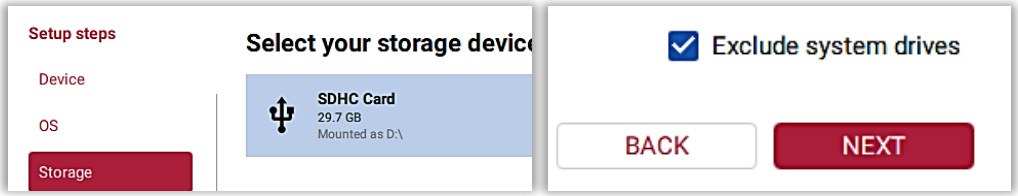


Figure 6.3: Selecting the microSD card as a storage device with Exclude System Drives turned on.

Step 4 – Customisation: Choose Username

After selecting the storage device, create a user account by entering the required login credentials, then click Next to continue.

- Username: xyz
- Password: xyz@123
- Confirm Password: xyz@123

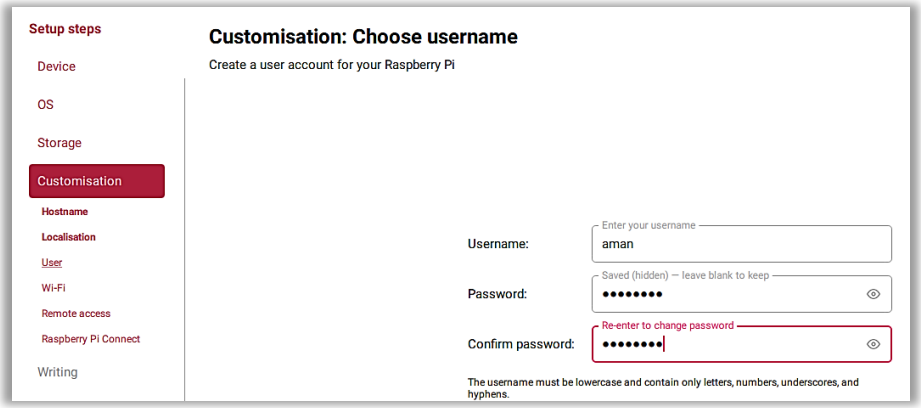


Figure 6.4: User account configuration in Raspberry Pi Imager.

Step 5: Choose Hostname

The next step is to configure the hostname to provide a unique identity for the Raspberry Pi on the local network, as shown in **Figure 6.**

- Hostname: xyz-pi

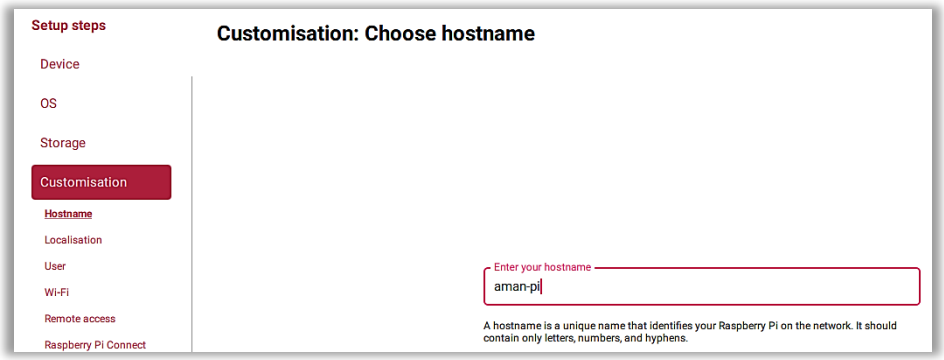


Figure 6.5: Hostname configuration in Raspberry Pi Imager.

Step 6: Choose Localisation

The next step is to configure the localisation settings by selecting the country, time zone, and keyboard layout, as shown in Figure 6.X.

- Country: India
- Time Zone: Asia/Kolkata
- Keyboard Layout: in

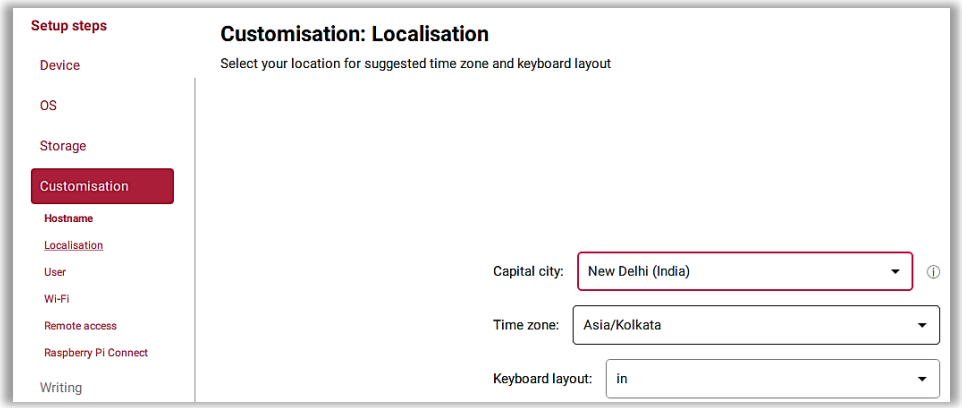


Figure 6.6: Localisation settings configured in Raspberry Pi Imager.

Step 7: Choose Username

The next step is to create a user account for the Raspberry Pi by specifying the username and password, as shown in **Figure 6.7.**

- Username: xyz
- Password: xyz@123
- Confirm Password: xyz@123

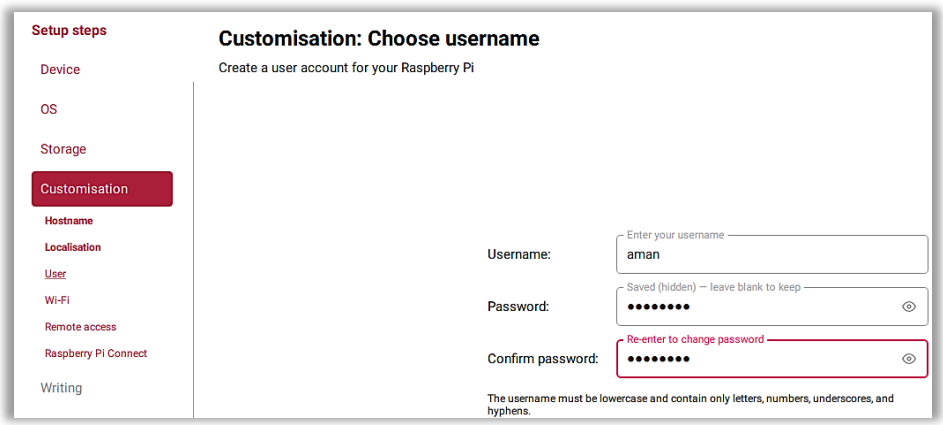


Figure 6.7: User account configuration in Raspberry Pi Imager.

Step 8: Choose Wi-Fi

The next step is to configure the Wi-Fi connection by entering the hotspot network name and password as shown in **Figure 6.8 below.**

- SSID: OPPO K14X 5G 227C
- Password: xyz@123
- Confirm Password: xyz@123

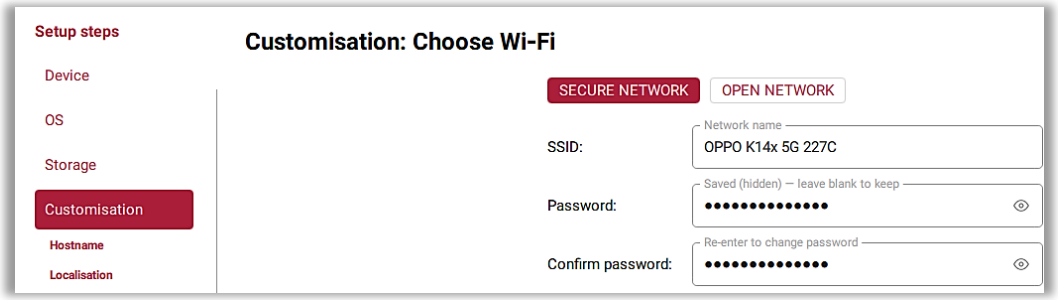


Figure 6.8: Wi-Fi configuration in Raspberry Pi Imager.

Step 9: Configure SSH Authentication

The next step is to **enable SSH** for remote access and select **Use password authentication** as the authentication mechanism, as shown in **Figure 6.9 below.**

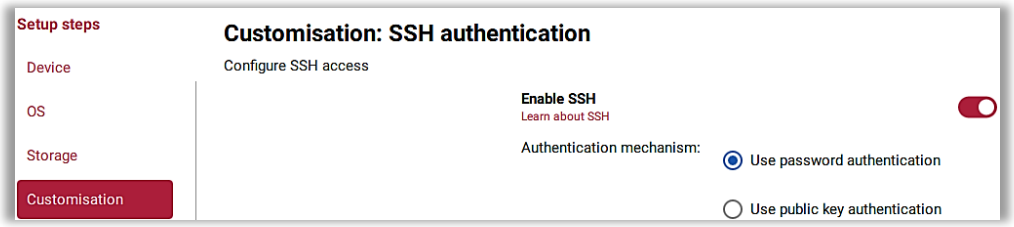


Figure 6.9: SSH authentication configuration in Raspberry Pi Imager.

Step 10: Configure Raspberry Pi Connect

The next task is to access Raspberry Pi Connect for remote access by logging in to get a token, selecting the Raspberry Pi Connect option, and opening the Raspberry Pi Connect window, as shown in **Figure 6.10 below.**

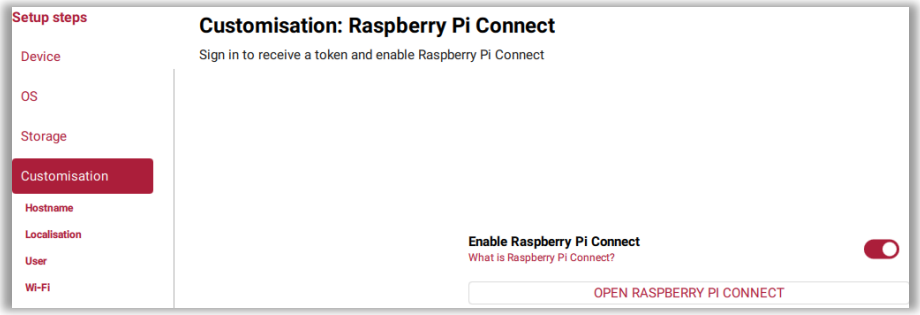


Figure 6.10: Raspberry Pi Connect configuration for remote access.

Step 11: Sign in with Raspberry Pi ID

The subsequent step is to **sign in with a Raspberry Pi ID**. In case of no account, you need to create one. Then, click Sign in as Aman Shukla (For reference) to proceed with creating the authentication key, as shown **below in Figure 6.11.**

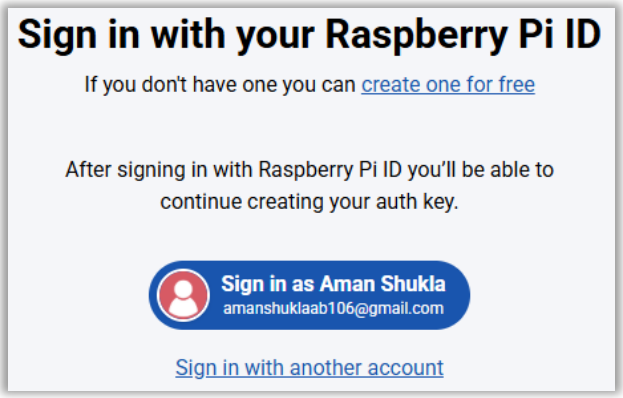


Figure 6.11: Raspberry Pi ID sign-in and authentication key setup.

Step 12: Write the Raspberry Pi OS Image

Review the selected device, OS, and storage device, then click Write to begin writing the Raspberry Pi OS image to the SD card, as shown in **Figure 6.12 below.**

- Device: Raspberry Pi 4/5
- OS: Raspberry Pi OS (64-bit)

- Storage: Apple SDXC Reader Media

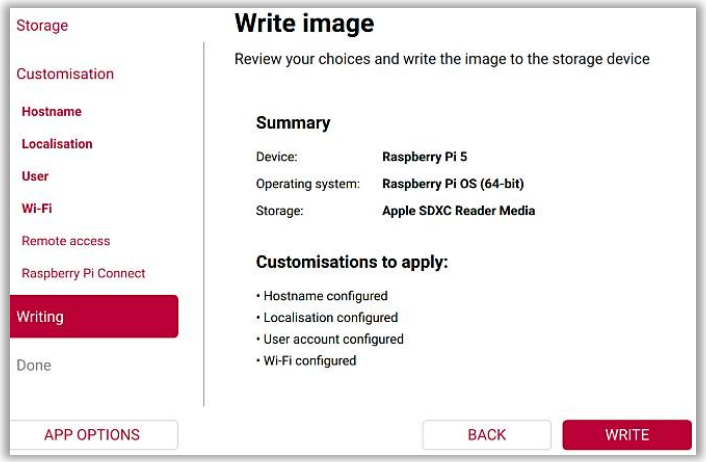


Figure 6.12: Raspberry Pi OS image writing confirmation and storage selection.

Step 12: Confirm Erase and Write

A confirmation message appears stating that all data on the selected SD card will be erased. Click "I UNDERSTAND, ERASE AND WRITE" to continue with the Raspberry Pi OS installation, as shown in **Figure 6.12 below.**



Figure 6.13: Confirmation to Erase and Write Raspberry Pi OS to the SD Card

Step 13: Write Image to SD Card

After confirming the erase and write operation, the Raspberry Pi OS image starts writing to the SD card. The progress can be monitored from the popup window; in the recorded setup, it was at 46%. The SD card was kept connected until the process was completed, as shown in **Figure 6.13 below.**

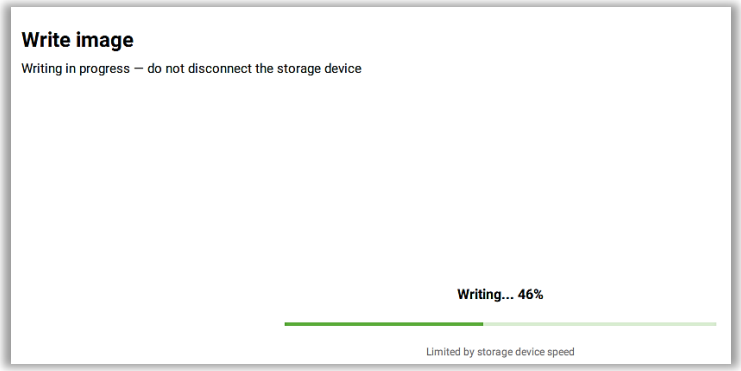


Figure 6.14: Raspberry Pi OS Image Writing Progress

6.3.2 Network and System Configuration

Step 1: Identify the Raspberry Pi IP Address

The Raspberry Pi IP address was identified using the following command in the terminal, as shown in Figure 6.14 below.

- Command: hostname -I
- Raspberry Pi IP Address: 10.38.4.1

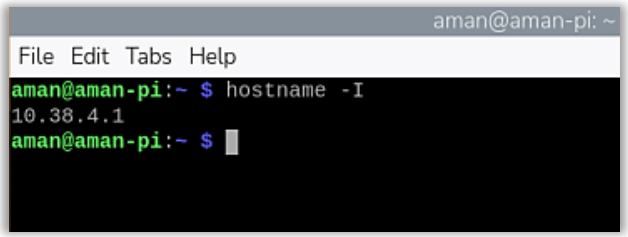


Figure 6.15: Raspberry Pi IP Address Identification Using hostname -I Command

Step 2: Enable VNC

From Interface Options, the **VNC option** was selected and enabled to allow graphical remote access to the Raspberry Pi, as shown in **Figure 6.16 below.**

- Command: sudoraspi-config

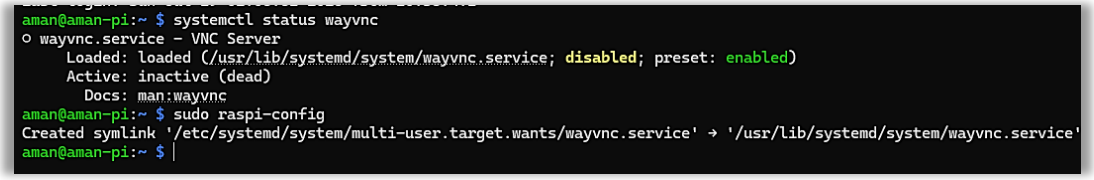


Figure 6.16: Enabling VNC Through Raspberry Pi Configuration

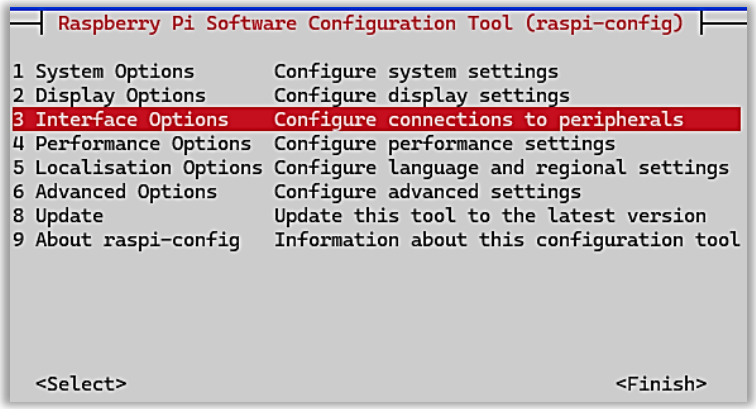


Figure 6.17: Raspberry Pi Configuration – Interface Options

In the next menu, select I3 VNC – Enable/disable graphical remote desktop access and press Enter, as shown in **Figure 6.17 below.**

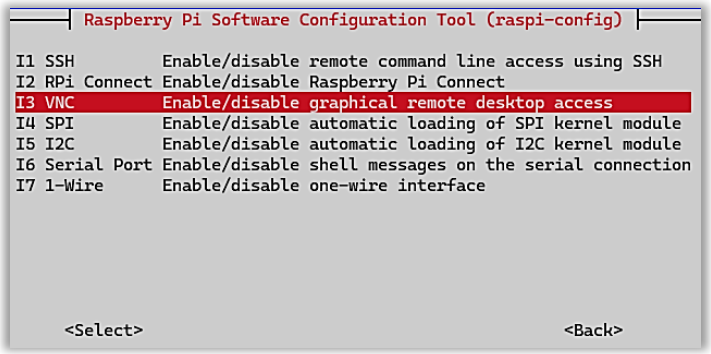


Figure 6.18: Selecting VNC from Interface Options

When the confirmation message "Would you like the VNC Server to be enabled?" appears, select **Yes** and press Enter to **enable VNC**, as shown in **Figure 6.18 below.**

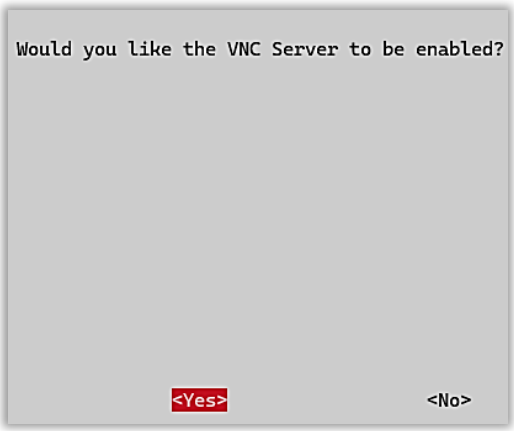


Figure 6.19: Enabling the VNC Server

6.3.3 Python Virtual Environment Setup

After verifying the installed Python environment, a separate virtual environment named yolo_env was created for the YOLO development setup.

Command:

- python3 -m venvyolo_env
- source yolo_env/bin/activate

The activated environment is indicated by **(yolo_env)** in the terminal prompt, as shown in **Figure 6.20**

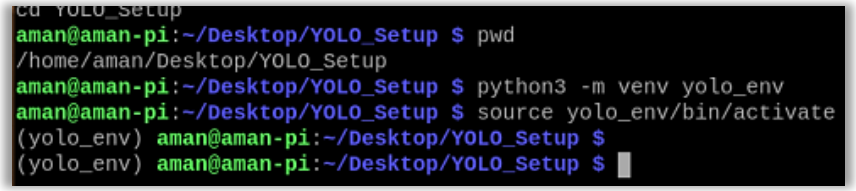


Figure 6.20: Creation and activation of the Python virtual environment

6.3.4 Deep Learning Library Installation

After the Python virtual environment was configured, the deep-learning libraries, plus the supporting Python packages, were installed on the Raspberry Pi.

Step 1: Upgrade pip

The pip package manager was upgraded to the latest version available.

Command:

pip install --upgrade pip

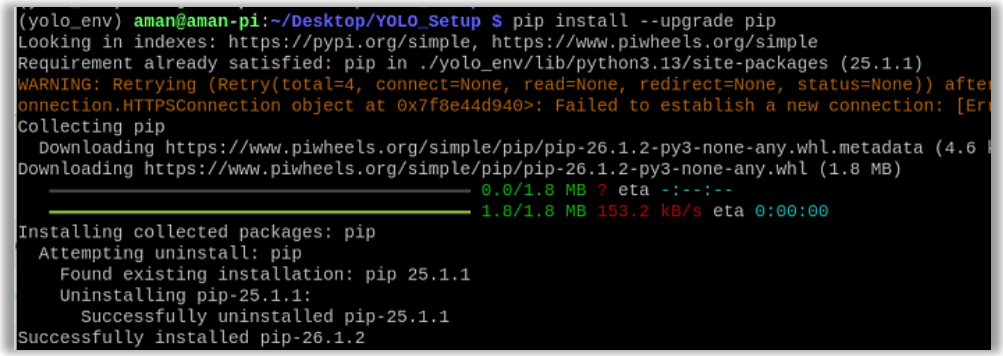


Figure 6.21: Upgrading pip in the Python Virtual Environment

Result:pip 26.1.2 was successfully installed, as shown in **Figure 6.21**

Step 2: Install PyTorch and TorchVision

PyTorch and TorchVision were installed on the raspberry pi to enable the raspberry pi have the necessary deep-learning framework.

Verified versions:

- PyTorch: 2.6.0+debian
- TorchVision: 0.21.0

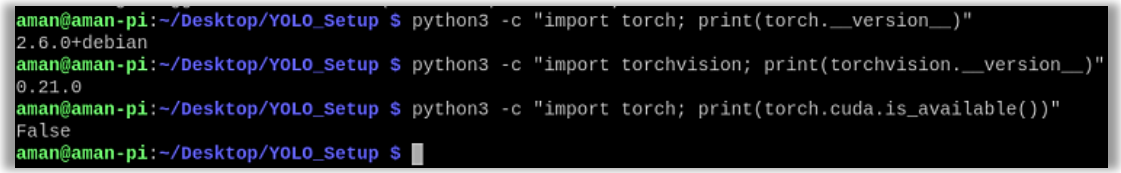


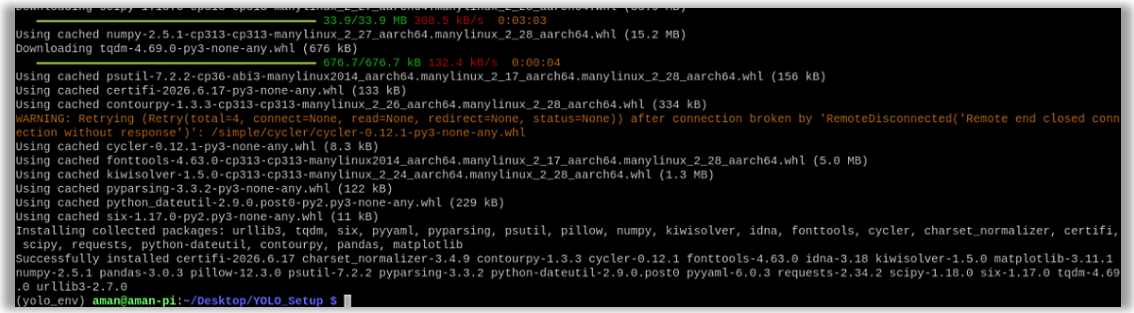
Figure 6.22: Installation of PyTorch and Torch Vision on Raspberry Pi

The installation and version verification are shown in **Figure 6.22**.

Step 3: Install Required Python Libraries

Supporting libraries including NumPy, Pandas, Matplotlib, SciPy, Pillow, PyYAML, Requests, tqdm, and psutil were installed for the development environment.

The successful installation is shown in **Figure 6.23.**



6.23: Installation of Required Python Libraries

Step 4: Verify Python Libraries

The installed libraries were verified using:

Command:

python -c "import pandas, matplotlib, PIL, yaml, requests, scipy, tqdm, psutil; print('Basic Libraries Installed Successfully')"

Result: Basic Libraries Installed Successfully

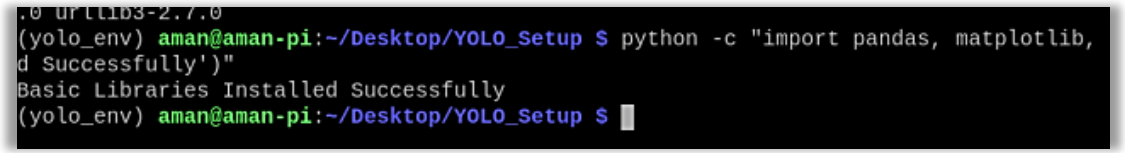


Figure 6.24: Verification of Installed Python Libraries

The verification is shown in **Figure 6.24.**

6.3.5 Development Environment Verification

After completing the software installation, the development environment was verified by checking the installed Python, PyTorch, and TorchVision versions.

Command:

- **Python version:**
- python3 –version

Response:

- Python 3.13.5

The Python version verification is shown in **Figure 6.25**

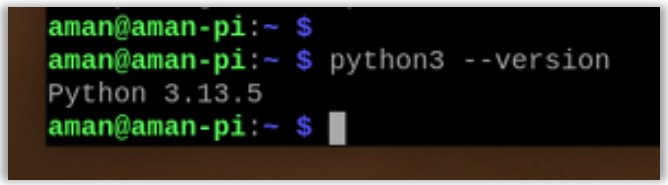


Figure 6.25: Python Version Verification

The deep-learning environment was further verified by checking PyTorch, Torch Vision, and CUDA availability:

Commands:

- python3 -c "import torch; print(torch.__version__)"
- python3 -c "import torchvision; print(torchvision.__version__)"

- python3 -c "import torch; print(torch.cuda.is_available())"

Response:

- 2.6.0+debian
- 0.21.0
- False

The verification **confirmed** that PyTorch and TorchVision were installed successfully and CUDA was unavailable on the Raspberry Pi, indicating CPU-based inference, as shown in **Figure 6.26.**

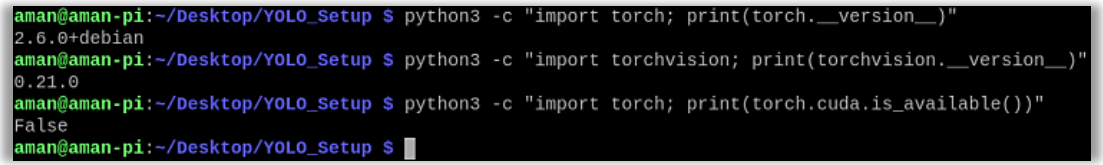


Figure 6. 26: Verification of PyTorch, TorchVision, and CUDA Availability

CHAPTER 7

DATSET AND DEEP LEARNING MODEL

7.1 Dataset Selection

The VisDrone dataset was selected for training the YOLO11n model because it contains aerial images captured from a drone-based, bird's-eye perspective and provides multiple object categories relevant to surveillance applications. The dataset was therefore suitable for developing and evaluating the proposed aerial object detection system.

The selected dataset contains ten object classes: pedestrian, people, bicycle, car, van, truck, tricycle, awning-tricycle, bus, and motor.

7.2 VisDrone Dataset

The **VisDrone2019 dataset** was used for training the object detection model because it contains aerial images captured using drone-mounted cameras and is suitable for drone-based surveillance applications. The dataset was downloaded from **Kaggle** for this project.

The dataset contains **10 object classes with corresponding Class IDs**: pedestrian (0), people (1), bicycle (2), car (3), van (4), truck (5), tricycle (6), awning-tricycle (7), bus (8), and motor (9). Each object is provided with corresponding bounding-box annotations.

For this project, the dataset was prepared for YOLO-based training using the **VisDrone.yaml** configuration file and organized into the required training and validation structure.

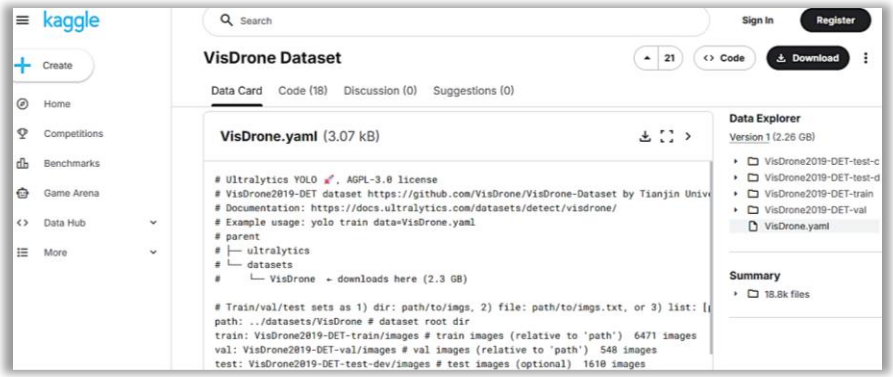


Figure 7.1: VisDrone Dataset Download Page on Kaggle

7.3 Dataset Preparation

After downloading and extracting the Zip folder from the Kaggle website, the downloaded dataset was saved in the project directory on the development system.

Dataset Working Directory:

C:\Users\amans\Desktop\Drone_AI_Project\Dataset

The dataset was then organized according to the YOLO training structure and prepared using the **VisDrone.yaml** configuration file.

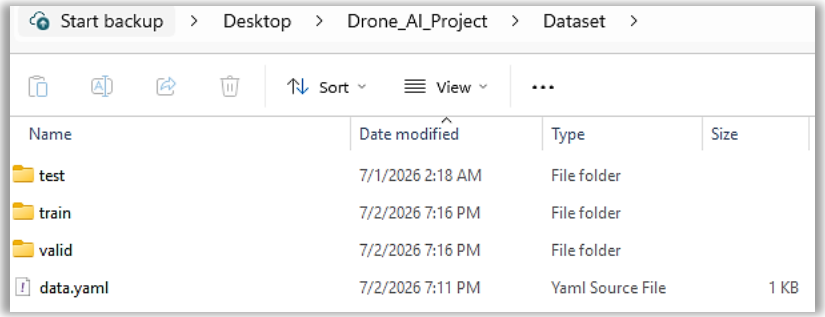


Figure 7.2: VisDrone Dataset Working Directory

7.4 YOLO11 Architecture

The object detection model used in this project is implemented using the **Ultralytics YOLO11n**. This family of models performs object detection using a single stage approach where an input image is processed through the network to produce the detected objects in the form of bounding boxes, classes, and confidence scores.

The model comprises three main modules:

Backbone: This block carries out the feature extraction process from the input image.

Neck: It merges backbone features from different layers to facilitate enhanced detection of objects of varying sizes

Detection Head: It employs the extracted and merged features to predict the final set of bounding boxes, class labels, and confidence scores.

In this project, the Ultralytics provides several variants of the YOLO model, including the YOLO11n (Nano). This version is ideal for the deployment and inference of the model on the raspberry pi 4 due to its small size.

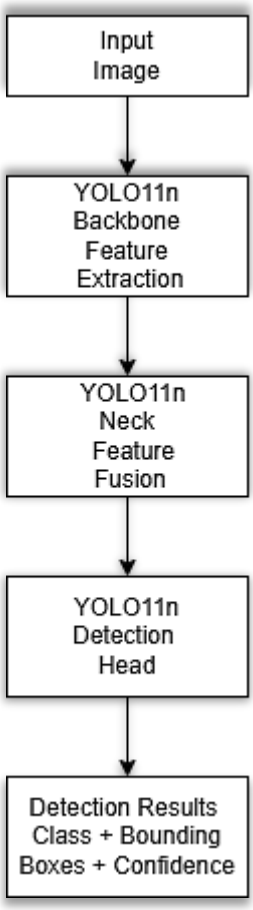


Figure 7.3: Simplified Architecture of the Ultralytics YOLO11n Model Used for Object Detection

Figure 7.3 shows the simplified architecture of the Ultralytics YOLO11n model used in this project. The input image is processed by the backbone for feature extraction, followed by feature fusion in the neck. The detection head then produces the object class, bounding box, and confidence score.

7.5 Selection of YOLO11n

The **YOLO11n (Nano)** model from **Ultralytics** was selected for this project because its lightweight architecture provides a suitable balance between object detection capability and computational requirements.

Since the final model was intended for inference on the Raspberry Pi 4, a smaller model was preferred to reduce the computational load on the edge device. Therefore, YOLO11n was selected for the subsequent training, fine-tuning, and deployment stages of the project.

7.6 Model Training and Fine-Tuning

The chosen **YOLO11n** model was trained and fine-tuned using the prepared **VisDrone-based aerial surveillance dataset**. The model development process was completed using the **Ultralytics YOLO framework** on a CPU. The model was **initially trained for 20 epochs**, followed by an additional **20 epochs of fine-tuning** process.

The purpose of this two-stage training approach was to first train the YOLO11n model on the given aerial dataset and then utilize the previously obtained best model for further training of the model's parameters.

7.6.1 Training Environment

The training experiments were performed in a Python virtual environment using the Ultralytics YOLO framework. The main software configuration used during the experiments was:

Table 7.1: Training Environment and Experimental Configuration

Parameter	Configuration
Deep Learning Model	YOLO11n
Framework	Ultralytics YOLO
Ultralytics Version	8.4.84
Python Version	3.14.0
PyTorch Version	2.12.1+cpu
Processing Device	CPU
GPU Memory	0G
Input Image Size	640 × 640 pixels
Batch Size	8
Data Loader Workers	0
Number of Classes	10

The model on which the experiment was conducted was trained on an **11th Gen Intel Core i5-1135G7 processor with a frequency of 2.40 GHz**. The software environment PyTorch, on which the program was implemented, uses a CPU for training, as evidenced by the entry GPU_mem = 0G in the training logs.

7.6.2 Initial Training of YOLO11n

The first step involved training the chosen YOLO11n model using the constructed dataset. The training process was carried out for **20 epochs** with an input image size of **640 × 640 pixels** and a batch size of **8**.

The training command:

yolo detect train model=yolo11n.pt data=Dataset\data.yaml epochs=20 imgsz=640 batch=8 device=cpu workers=0 project=runs\detect name=DroneSurveillance_YOLO-2

The command sets the YOLO11n pretrained model as the initial model, the prepared data configuration as data.yaml, and the number of training epochs to 20. The image size was set to 640 × 640 pixels. The experiment was run on a CPU, so device=cpu was set. Finally, the number of data-loader workers was set to zero.

During training, the dataset was divided into batches and the model parameters were updated iteratively. The training process reported **809 iterations per epoch**. Therefore, for 20 epochs, the approximate number of training iterations was:

809 × 20 = 16,180 iterations

The training process monitored three major loss components:

- **Box Loss**: refers to the error introduced while localizing the predicted boxes,
- **Classification loss (cls_loss):** is the error due to incorrect classification, while
- **the Distribution focal loss (dfl_loss):** aids precise localization of the bounding-box coordinates.
- These losses were tracked throughout the training process to observe how the parameters evolved with each training step.

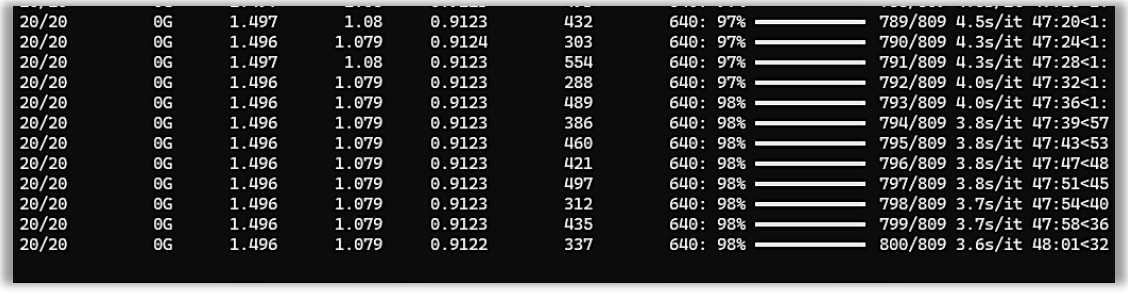


Figure 7.4 shows the training progress during the initial 20-epoch YOLO11n training process.

7.6.3 Training Progress and Final Epoch

During the preliminary training process, the values of box loss, classification loss, and DFL loss were updated as the model was trained using the labeled aerial images. Thus, the given training log below demonstrated the model's learning process for each training batch. According to the final line of the training log, during the last epoch of the preliminary training, the input image size was set to **640 × 640 pixels**, **while the losses were the following:**

- **Box Loss:** 1.496
- **Classification Loss:** 1.079
- **DFL Loss:** 0.912

Furthermore, the training log below showed that 0G GPU memory was used for training, which implies that the CPU was used instead of GPU. After preliminary training, the model weights were saved in the best.pt and last.pt files, where the best model was used for the next stage of the process.

7.6.4 Fine-Tuning of YOLO11n

After completing the initial training for 20 epochs, the **best.pt** file was utilized for continued fine-tuning. The purpose of this step was to further adapt the already pretrained YOLO11n model to the prepared dataset of aerial surveillance images. Rather than using the original pretrained weights, the best model from the first stage of training was used as a starting point for the second stage of training.

The fine-tuning command used was:

yolo detect train model="runs/detect/DroneSurveillance_YOLO-2/weights/best.pt" data="Dataset/data.yaml" epochs=20 imgsz=640 batch=8 device=cpu workers=0 project="runs/detect" name="DroneSurveillance_Finetune"

The important parameters used during fine-tuning were:

Table 7.2: Configuration Parameters Used for YOLO11n Fine-Tuning

Parameter	Value
Starting Model	Previous best.pt
Dataset Configuration	Dataset/ data.yaml
Fine-Tuning Epochs	20

Image Size	640 × 640 pixels
Batch Size	8
Processing Device	CPU
Workers	0
Output Directory	Drone Surveillance Finetune

Before starting fine-tuning, the model successfully transferred the previously learned parameters. The training log reported:

Transferred 499/499 items from pretrained weights

This indicates that the learned model parameters from the previous training stage were successfully loaded into the fine-tuning model.

The dataset scanning stage identified:

- **Training images:** 6,471
- **Validation images:** 548
- **Background-only images:** 0
- **Corrupt images:** 0

A small number of duplicate labels were also identified and automatically removed during dataset preparation.

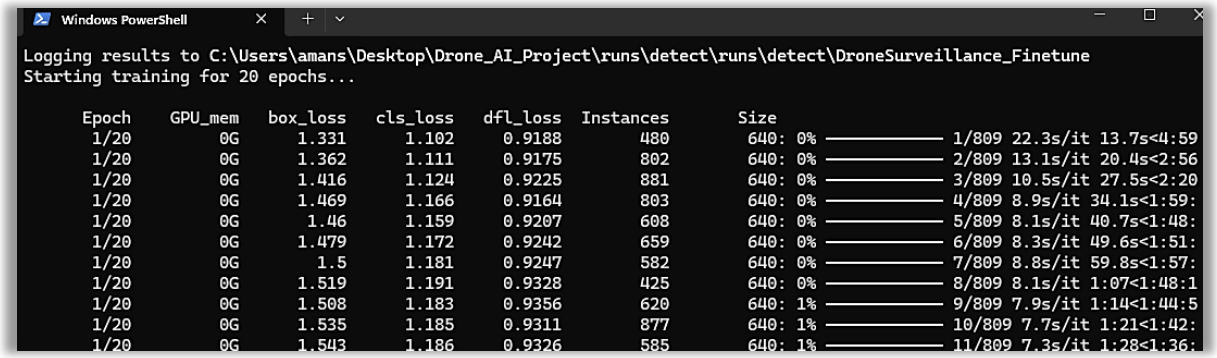


Figure 7.5 presents the command and initialization of the YOLO11n fine-tuning experiment.

7.6.5 Completion of Fine-Tuning

The fine-tuning process was continued on the CPU for another 20 epochs. Overall, the fine-tuning took about **20.139 hours.**

As can be seen in the final line below, during the last epoch, the process achieved approximately:

- **Box Loss:** 1.445
- **Classification Loss:** 1.010
- **DFL Loss:** 0.9006

Finally, the last stage of training is complete, having finished all **20/20 epochs**, and the model weights are ready to be saved.

As a result, the fine-tuning process yielded two model files:

- **last.pt** – model weights from the final training state.
- **best.pt** – best-performing model weights selected during the training process.

The obtained information about the optimizer was deleted from the saved checkpoints; thus, the size of each model file became approximately **5.4 MB**.

The best model, which was obtained as a result of fine-tuning, is represented by the best.pt checkpoint and was used for further validation and implementation.

7.6.6 Validation of the Fine-Tuned Model

After completing training, the best model was validated using the validation part of the prepared dataset, which consisted of

- 548 images
- 38,759 annotated object instances

The validation process was performed using the CPU-based PyTorch environment. The final overall validation results obtained from the fine-tuned model were:

Table 7.3: Validation Performance Metrics of the Fine-Tuned YOLO11n Model

Metric	Value
Precision (P)	0.389
Recall (R)	0.319
mAP@50	0.281
mAP@50–95	0.157

The model performance was assessed using the **Precision** and **Recall** metrics. As a result, for the YOLO11n model, the Precision value of 0.389 reflects the ratio of detected objects to all objects in

the image, while the Recall value of 0.319 reflects the ratio of detected objects to all annotated objects.

As a result of the evaluation, an**mAP@50** value of 0.281 was obtained, which represents the mean average precision at an IoU threshold of 0.5. An**mAP@50–95** value of 0.157 was obtained, which represents the average precision at various IoU thresholds (from 0.5 to 0.95).

Finally, the model fusion process yielded the final YOLO11n model configuration with approximately **2.58 million parameters** and **6.3 GFLOPs**.

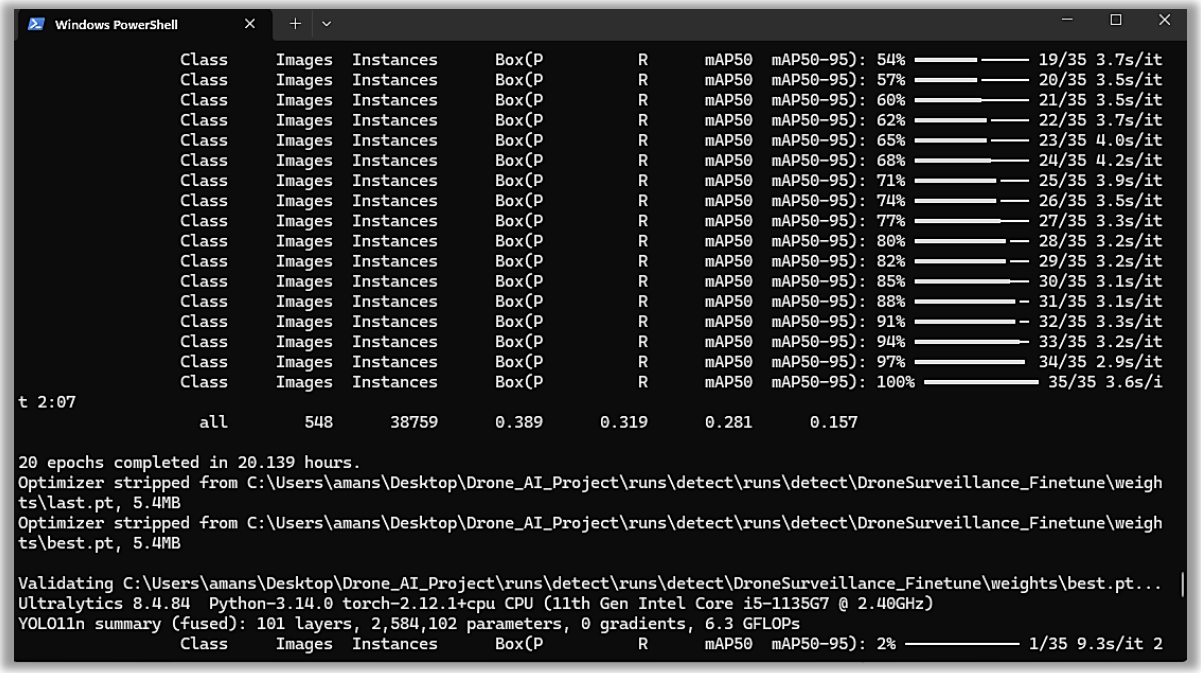


Figure 7.6: Completion of YOLO11n fine-tuning and validation results of the final model

7.7 Transfer of Fine-Tuned Model to Raspberry Pi

Finalizing the transfer of the fine-tuned YOLO11n best.pt weights from the Windows machine to the **Raspberry Pi 4** for edge deployment, where the model will be utilized for its primary inference objective. The file transfer occurred over the local network using the **Secure Copy Protocol (SCP)**.

Below is the command used to copy the best.pt weights file from the Windows machine to the Raspberry Pi 4:

scp"C:\Users\amans\Desktop\Drone_AI_Project\runs\detect\runs\detect\DroneSurveillance_Finetune\weights\best.pt"aman@10.38.4.1:/home/aman/Desktop/YOLO_Model/

The transfer was completed successfully, with the **5.2 MB best.pt** file copied to the Raspberry Pi.



Figure 7.7: Transfer of the fine-tuned YOLO11n model to Raspberry Pi using SCP

7.7.1 Verification of the Deployed Model

After the transfer, the directory's directory listing was checked using the ls -lh command. The target directory contained the fine-tuned **best.pt model**, the YOLO11n model, camera inference script, and test image.

The fact that the best.pt file occupying about **5.2 Mb** was present on the Raspberry Pi indicated that the model transfer was successful.

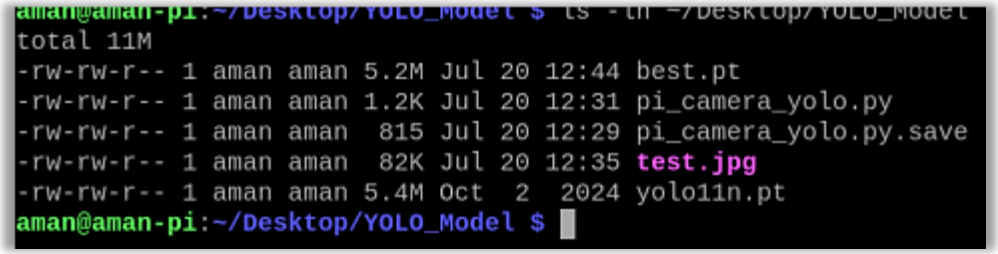


Figure 7.8: Verification of YOLO11n model files on Raspberry Pi

7.7.2 Model Inference on Raspberry Pi

The model was tested on the **Raspberry Pi CPU** using an input image, with an inference time of **1450.6 ms (1.45 s) per image**, while preprocessing and postprocessing took approximately **17.5 ms** and **5.5 ms**, respectively.

The inference results demonstrated that one would be able to load the fine-tuned YOLO11n model onto the Raspberry Pi and run it successfully to conduct **object detection on the Edge** using the lightweight model.

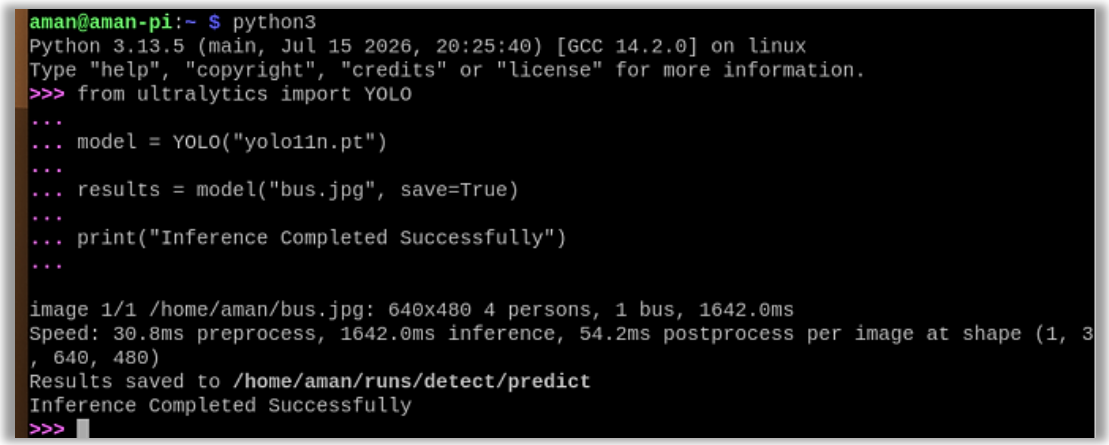


Figure 7.9: YOLO11n inference performance on Raspberry Pi CPU

CHAPTER 8

MODEL EVALUATION AND RESULTS

8.1 Evaluation Metrics

I have used standard metrics for object detection, including **Precision, Recall, mAP@50, and mAP@50–95**, to evaluate the performance of my fine-tuned YOLO11n model. Using these metrics, I have calculated the overall performance of my model on the validation data, which consists of **548 validation images, 38,759 instances of objects. The results I have got are listed below:**

Table 8.1: Overall Evaluation Metrics of the Fine-Tuned YOLO11n Model

Metric	Value
Precision	0.389
Recall	0.319
mAP@50	0.281
mAP@50–95	0.157

The Precision–Recall curve in **Figure 8.1** shows the detection performance of the model for the different object classes. Among the evaluated classes, the car class achieved the highest AP of 0.713, while bicycle showed a lower AP of 0.047. The overall mAP@50 was 0.281, indicating that the model was able to detect several aerial object categories, although performance varied across classes.

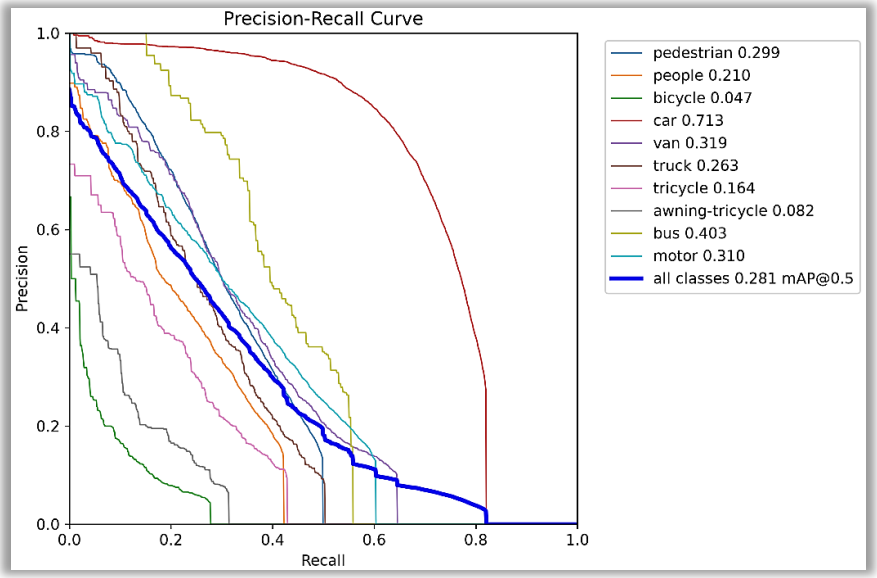


Figure 8.1: Precision–Recall Curve of the Fine-Tuned YOLO11n Model

8.2 Model Performance

The fine-tuned YOLO11n model resulted in an overall **Precision of 0.389** and **Recall of 0.319** on the validation set. In addition, the model achieved an overall **mAP@50 of 0.281** and **mAP@50–95 of 0.157.**

The results for individual classes were varied, with the car class being the best-performing one, having an AP of 0.713. The second one was bus with an AP of 0.403, and the performance for such classes as bicycle and awning-tricycle was significantly lower at 0.047 and 0.082, respectively

The results suggest that the fine-tuned model was able to identify many of the typical objects presented on the images, but its performance substantially varied depending on the category and its visual appearance.

8.3 Confusion Matrix

The confusion matrix below was obtained by analysing class-wise prediction results of the fine-tuned model **(YOLO11n)**. The matrix was derived from the validation dataset, where **true classes** of objects were compared to predicted ones. Background is included as one of the categories.

It can be seen that most of the car instances **(9,964)** were predicted correctly along with **pedestrians (2,677), people (702) and motors (1,171).** Predictions also show some confusion between similar classes such as car and van (834 van instances were predicted as cars). Other classes were either predicted as background incorrectly or predicted correctly, but their occurrences were low. In particular, the number of instances of a particular class that were predicted as background (False Negative) was significantly higher for **pedestrian (2,198), people (2,049), bicycle (1,295), and motor (518)**. Thus, the analysis of the confusion matrix shows that fine-tuned YOLO11n has some issues distinguishing between particular classes. However, most of the frequently occurring classes were predicted with high accuracy.

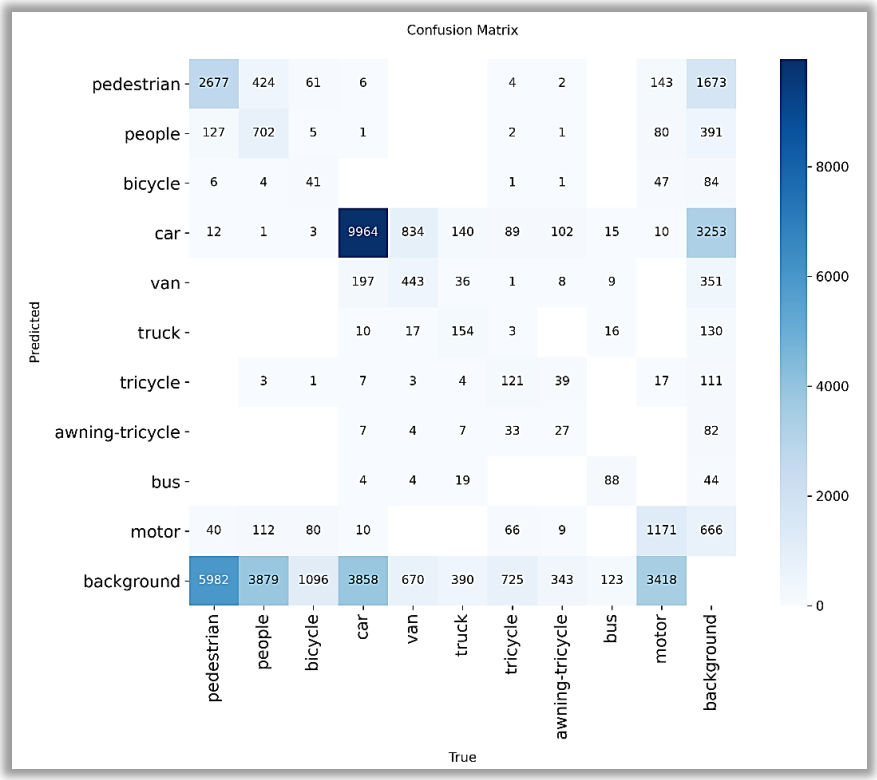


Figure 8.2: Confusion Matrix of the Fine-Tuned YOLO11n Model

8.4 Training Curves

The training curves were plotted to visualize the training loss and validation performance during the fine-tuning process of YOLO11n for 20 epochs. Box loss, classification loss, DFL loss, Precision, Recall, mAP@50, and mAP@50–95 are demonstrated for each training epoch.

According to the plot above, the loss decreased during the training process with a few exceptions for validation losses. In particular, the box loss decreased from 1.59 to 1.44, while the classification loss decreased from 1.17 to 1.01. Additionally, DFL loss reduced from approximately 0.924 to 0.901 during the 20 epochs. On the other hand, the validation loss also decreased during the training process.

Other performance metrics demonstrated a similar pattern with slight fluctuations during the training process. For instance, Precision, Recall, mAP@50, and mAP@50–95 reached 0.389, 0.319, 0.281, and 0.157, respectively. Altogether, the visual analysis of the training curves suggests that the model was fine-tuned properly without overfitting since the loss remained consistent for both validation and training sets during the 20 epochs.

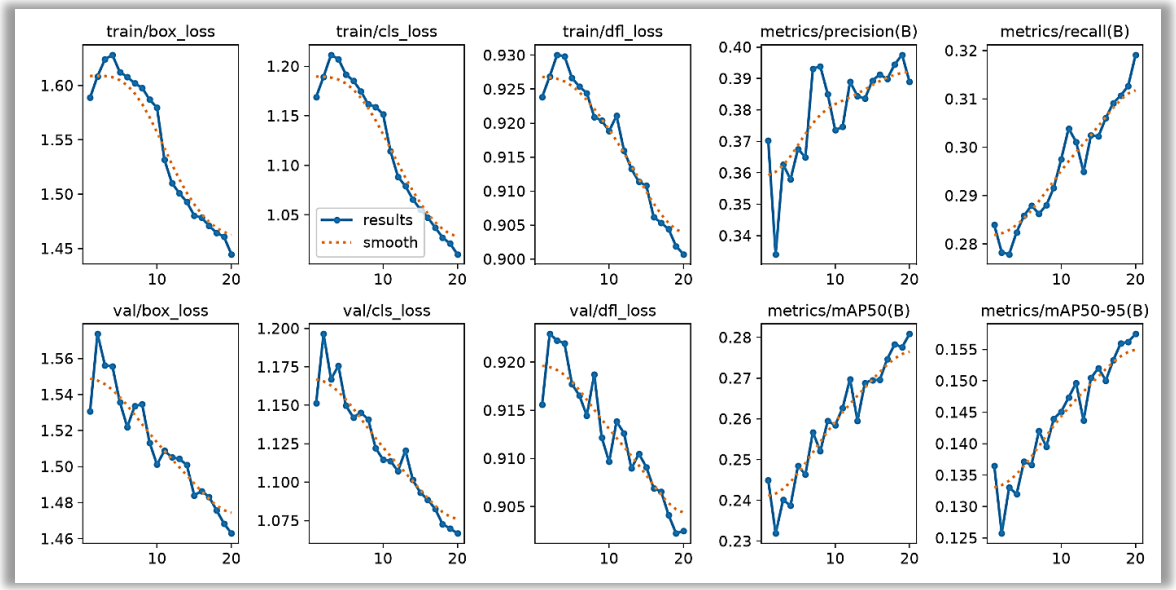


Figure 8.3: Training and Validation Curves

8.5 Object Detection Results

The object detection accuracy of the fine-tuned model was evaluated on the validation images from the VisDrone data set. The model detected objects in the images and predicted their class labels along with the confidence scores as shown below. The bounding boxes with corresponding class labels and confidence scores for one of the test images are presented in **Figure 8.5**. As can be seen, the model can successfully identify objects of different classes, including pedestrians, cars, vans, trucks, motorcycles, bicycles, and tricycles. The confidence scores vary with the sizes of the objects detected in the image as can be seen from this figure.

Thus, the fine-tuned model can be used for multi-class object detection of aerial images. However, as seen from this figure, there are some false detections, especially for small objects.

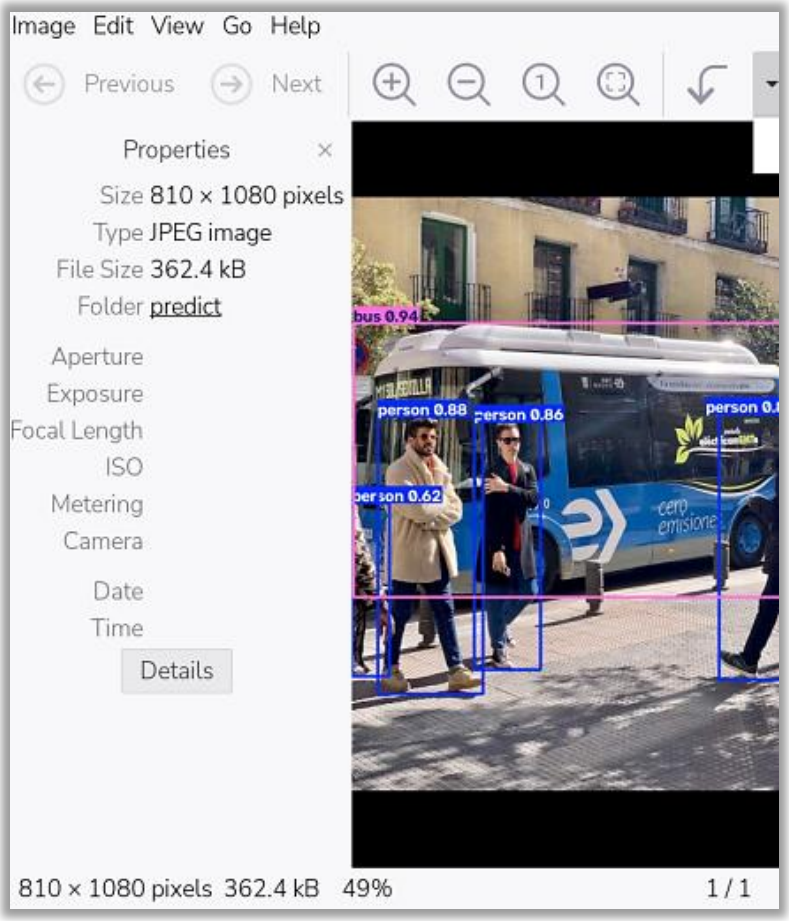


Figure 8.4: Object detection results using fine-tuned YOLO11n

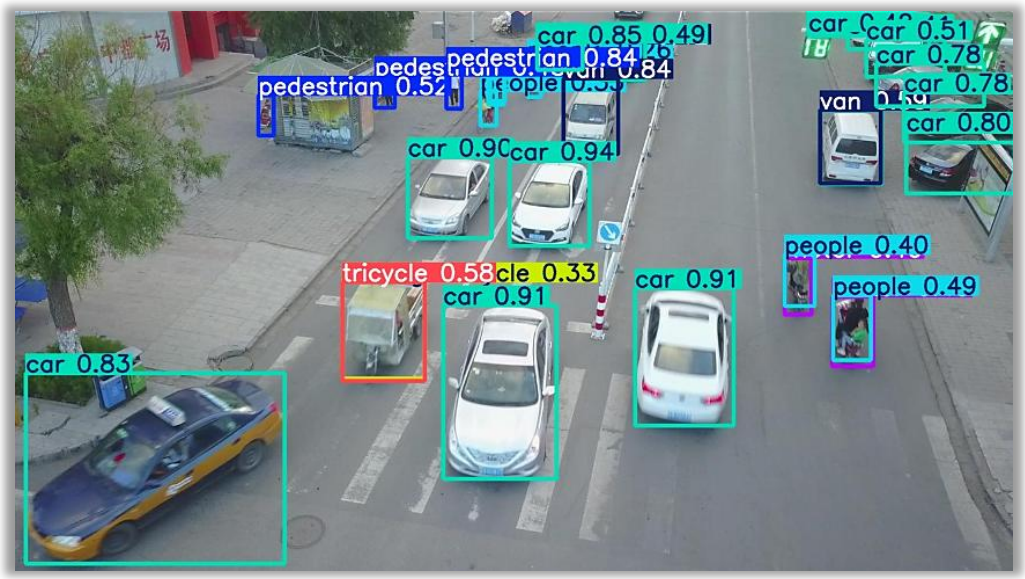


Figure 8.5: YOLO11n Object Detection Result on a VisDrone Aerial Image

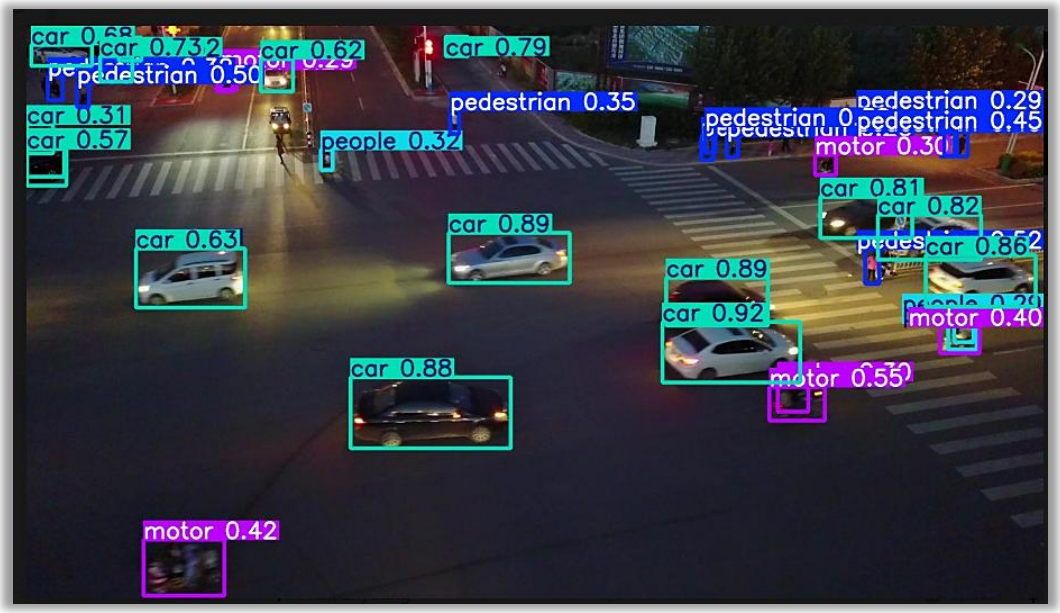


Figure 8.6: YOLO11n Object Detection Result in a Low-Light VisDrone Scene

Figure 8.6 also demonstrates the effect of challenging illumination conditions, where several small or distant objects were detected with lower confidence compared with the relatively clearer daytime scene shown in Figure 8.5

8.6 Analysis of Results

The tuned YOLO11n model did a good job of finding many different objects in aerial pictures from the VisDrone dataset. When we looked at the training curves we saw that the losses for training and validation got smaller over time. The Precision Recall, mAP@50 and mAP@50–95 values all got better as we fine-tuned the model.

The YOLO11n model got a Precision of 0.389 Recall of 0.319 mAP@50 of 0.281 and mAP@50–95 of 0.157 on the validation dataset. When we looked at the Precision and Recall for each object category we saw some differences. The YOLO11n model was really good at finding cars with an AP of 0.713. It was not as good at finding bicycles or awning-tricycles with AP values of 0.047 and 0.082 respectively.

The confusion matrix showed us that the YOLO11n model did well with objects it saw a lot, like cars and pedestrians. It got confused between objects that look similar like cars and vans. The YOLO11n model also had trouble finding objects, like pedestrians or bicycles that were far away or hard to see.

We can see some examples of how the YOLO11n model works in Figures 8.4–8.6. In the daytime picture the YOLO11n model found different objects. In the low-light picture the YOLO11n model still found some objects. It was not as confident and made some mistakes.

Overall the YOLO11n model is a starting point for finding many different objects, in aerial pictures from the VisDrone dataset. We can still improve it especially for small objects objects it does not see often and objects that look similar. To make the YOLO11n model better we could use training data try different ways of augmenting the data optimize the model and adjust the training parameters. This would be important if we want to use the YOLO11n model for time aerial surveillance. The YOLO11n model needs to be improved before we can use it for this.

CHAPTER 9

RASPBERRY PIDEPLOYMENTAND EDGE INFERENCE

9.1 Raspberry Pi Camera Module V3 Configuration

The Raspberry Pi Camera Module V3 was set up and tested prior to being attached to the YOLO11n algorithm through the inference pipeline. In this process, the Raspberry Pi Camera Module V3 was attached to the Raspberry Pi 4 via the CSI interface and was tested with the Raspberry Pi Camera Module V3 utility.

Step 1: Check camera detection

The camera configuration was first checked using:

Command: rpicam-hello

The preview window confirmed that the camera was functioning correctly. The terminal output also showed the camera being initialized through **libcamera v0.7.1+rpt202606**.

Step 2 — Verify Camera Stream

During initialization, the camera stream was configured at:

1296 × 972

The terminal output confirmed the selected camera and stream configuration:

- configuring streams: (0) 1296x972
- Selected unicam format: 1296x972

Step 3 — Verify Live Image

The camera preview displayed the live scene, including a cardboard box labeled **"Tyrone"**, confirming successful image acquisition from the connected camera.

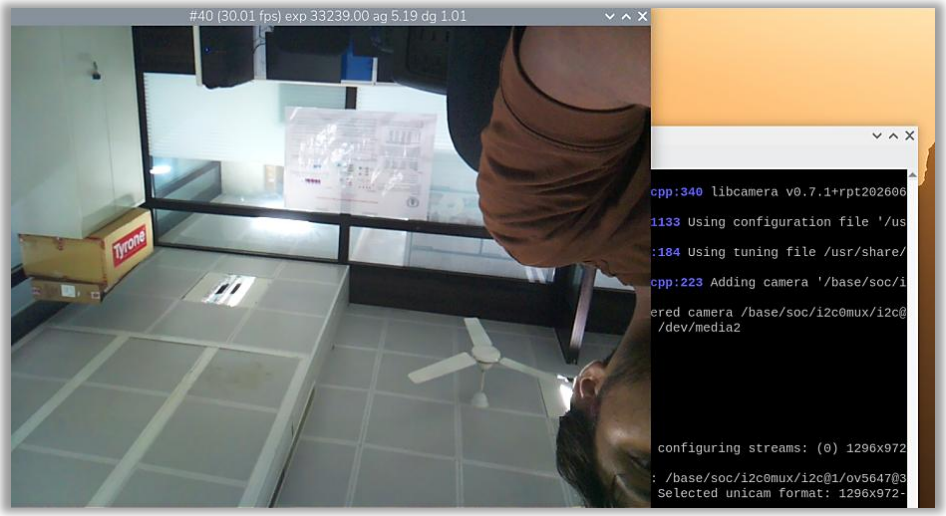


Figure 9.1: Raspberry Pi Camera Module V3 Preview and Verification

The camera was therefore successfully verified and was ready to provide image input for the YOLO11n inference pipeline described in the subsequent sections.

9.2 YOLO11n Model Deployment and Inference Setup

The fine-tuned YOLO11n model after training was deployed on the Raspberry Pi 4 for the purpose of edge-based inference. The best.pt model from the training processon Windows was transferred to theraspberry pi using the secure copy protocol.

Step 1 - Deploying the Fine-tuned Model

The trained model was transferred from windows using the following command.

Command:

Scp"C:\Users\amans\Desktop\Drone_AI_Project\runs\detect\runs\detect\DroneSurveillance_Finetune\weights\best.pt" aman@10.38.4.1:/home/aman/Desktop/YOLO_Model/

The transfer completed successfully, with the terminal showing best.pt transferred at 100%.

Step 2 — Verify Model Files

After transferring the model, the Raspberry Pi directory was checked using:

Command:

- cd ~/Desktop/YOLO_Model
- ls -lh

The directory contained the fine-tuned model:

- best.pt 5.2M
- yolo11n.pt 5.4M
- pi_camera_yolo.py
- test.jpg

The presence of best.pt confirmed that the fine-tuned YOLO11n model was **successfully deployed** on the Raspberry Pi.

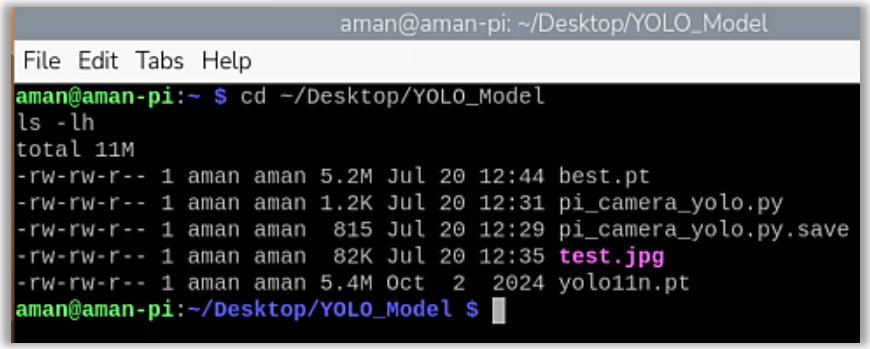


Figure 9.2: Verification of the Fine-Tuned YOLO11n Model on Raspberry Pi

Step 3 - Prepare the Inference Program

The YOLO inference program was prepared in the deployment directory using:

Command: nano pi_camera_yolo.py

The Python code was set up to load the YOLO11n model and run the program to detect objects in images. The obtained script was further tested on the Raspberry Pi 4.

9.3 Inference Pipeline

The inference pipeline was designed to load the best trained **YOLO11n model (best.pt),** take an input image, perform the object detection on a raspberry pi 4 CPU, and save the resulting image with detected boxes.

Step 1 — Open the YOLO Model Directory

The deployment directory containing the model and inference script was accessed using:

Command: cd ~/Desktop/YOLO_Model

Step 2 — Load the Fine-tuned YOLO11n Model

The fine-tuned model was loaded using the Ultralytics YOLO framework.

Command:

- from ultralytics import YOLO
- model = YOLO("best.pt")

The successful loading of the model showed that the transferred **best.pt file** was compatible with the Ultralytics environment installed on the raspberry pi.

Step 3 - Image Inference

The model was given an input image to make predictions on, for object detection.

Command: results = model ("test.jpg", save=True)

The model returned the detected image with the detected classes and their corresponding bounding boxes and confidence scores.

Step 4 — Save and Verify the Detection Result

The generated detection image was saved automatically by the Ultralytics inference pipeline. The output directory was then checked using:

Command: find runs -name "*.jpg"

The resulting image was stored in the corresponding runs/detect/ prediction directory.

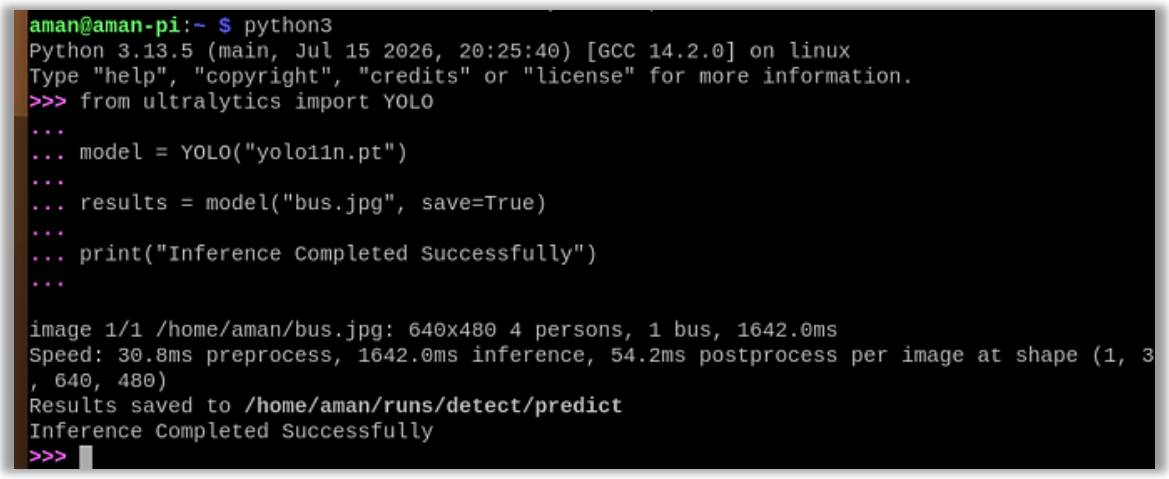


Figure 9.3: YOLO11n Inference Pipeline Execution on Raspberry Pi

Overall, the inference process demonstrated that the YOLO11n model could be loaded on the Raspberry Pi, process an input image, detect the objects present in the scene, and generate the corresponding bounding-box and confidence-score output.

9.4 Object Detection on Raspberry Pi

Following the model inference, I performed object detection of the captured image by the Raspberry Pi Camera Module V3 using the YOLO11n model. On the output, detected by the model as well as its confidence.

Step 1 - Provide the Camera Captured Image

The image captured using the Raspberry Pi Camera Module V3 was stored as test.jpg on the Raspberry Pi and provided as input to YOLO11n for inference.

Step 2 - Perform Object Detection

YOLO11n processed the input image and provided the detection results as output, which included the detected object class, bounding box, and confidence score.

Step 3 - Verify the Detection Results

The detection result was then opened to confirm the detection results. As can be seen in Figure 9.4, the model detected a person with a confidence level of 0.59 (59%) and displayed the corresponding detection bounding box around the person.

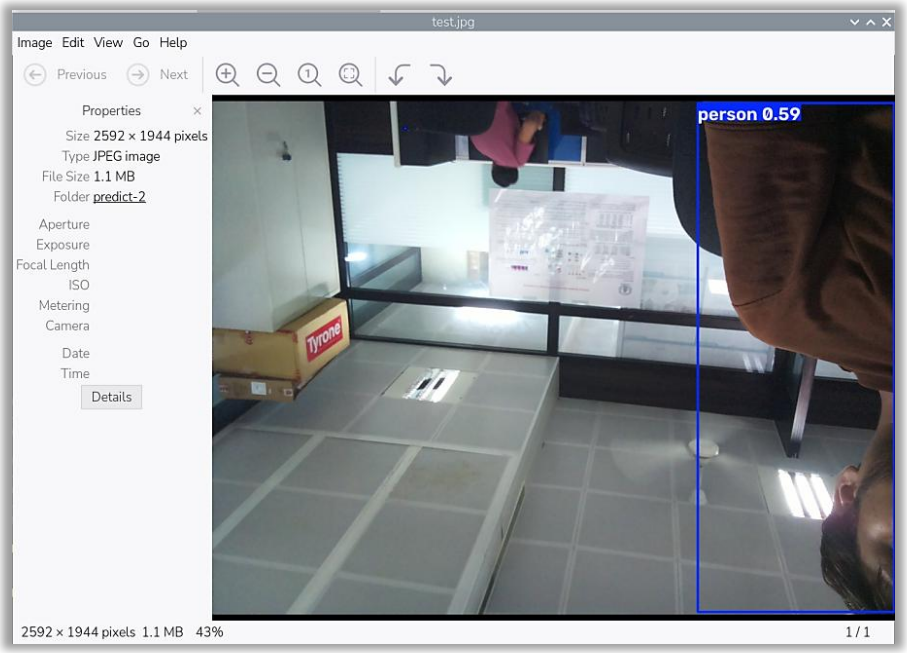


Figure 9.4: YOLO11n Object Detection Result on Raspberry Pi

The result showed that the YOLO11n model managed to process the image captured by the camera on the Raspberry Pi and output a result with the predicted class and confidence.

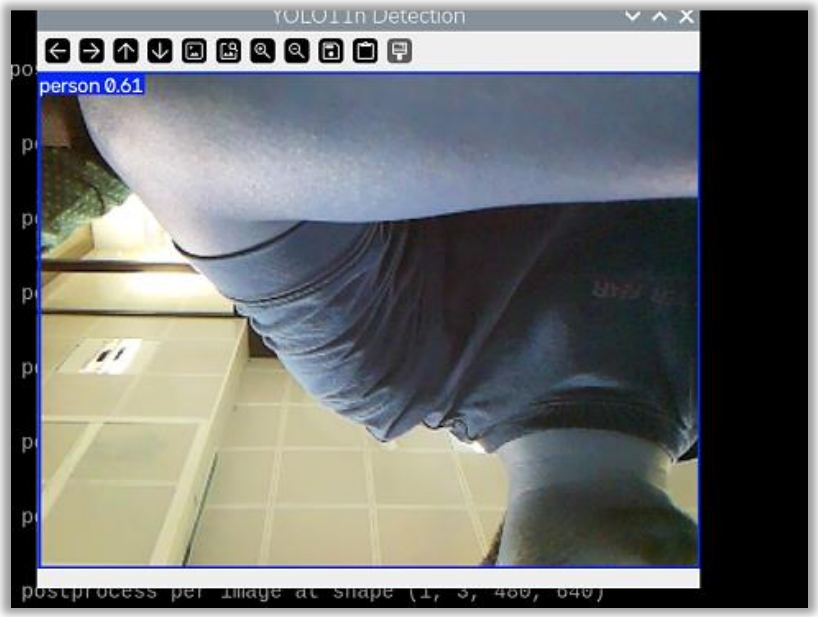


Figure 9.5: Live YOLO11n Object Detection Using Raspberry Pi Camera Module V3

Besides testing the camera image, the system was also tested with the live camera stream. The YOLO11n model processed the camera images and output the detected object with the confidence, as shown in figure 9.5. This test showed how the camera stream is processed through the model directly on the raspberry pi.

9.5 Edge Inference Performance and Observations

The performance of the YOLO11n model was tested on the Raspberry Pi 4to assess its performance on the actual edge device. This test was performed using CPU inference on the camera image.

The inference process achieved the following performance per image:

- Preprocessing: 17.5 ms per image
- Inference: 1450.6 ms per image
- Postprocessing: 5.5 ms per image
- Total processing time: 1473.6 ms (approximately 1.47 seconds) per image
- Effective processing rate: approximately 0.68 FPS

to assess its performance on the actual edge device and demonstrating its ability to perform object recognition to assess its performance on the actual.to assess its performance on the actual proved the possibility of deploying the trained YOLO11n model on a Raspberry Pi-based camera, creating the basis for an autonomous visual detection and surveillance system. Onboard artificial intelligence would allow such a system to perform real-time processing of visual information without the need for additional computing resources.

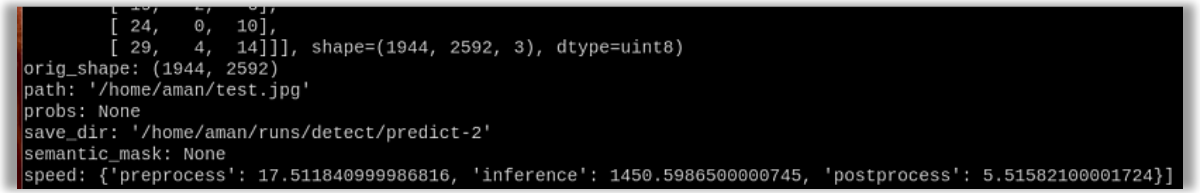


Figure 9.6: YOLO11n Inference Performance Metrics on Raspberry Pi CPU

CHAPTER 10

COMPLETE SYSTEM INTEGRATION

10.1 Integrated System

This section describes the complete system comprising the S550 hexacopter, Pixhawk flight controller, Raspberry Pi 4, Raspberry Pi Camera Module V3, and the YOLO11n-based object detection system. The components established and tested in the preceding chapters were combined to formulate this system, as illustrated below.

The S550 hexacopter forms the aerial frame while the Pixhawk manages the flight control systems. The Raspberry Pi 4 computer is installed and serves as the edge-computing layer and is connected to the Raspberry Pi Camera Module V3. This camera module captures images that are fed into the YOLO11n model for object detection.

When in use, the camera captures visual data that are relayed to the Raspberry Pi computer for processing. The computer then executes the YOLO11n model to detect and identify objects in the image, labeling them with their respective confidence scores. The final detection results can be viewed in the interface provided below.

The combined system described above was formulated to establish the complete processing pipeline, whereby images acquired from the aerial frame are handled through an onboard edge-computing system to execute object detection successfully. It is evident that the system facilitates the implementation of an onboard edge-AI surveillance system that does not require a desktop computer to execute the inference process.

The final composite system forms the basis of the end-to-end processing workflow described in the following section.

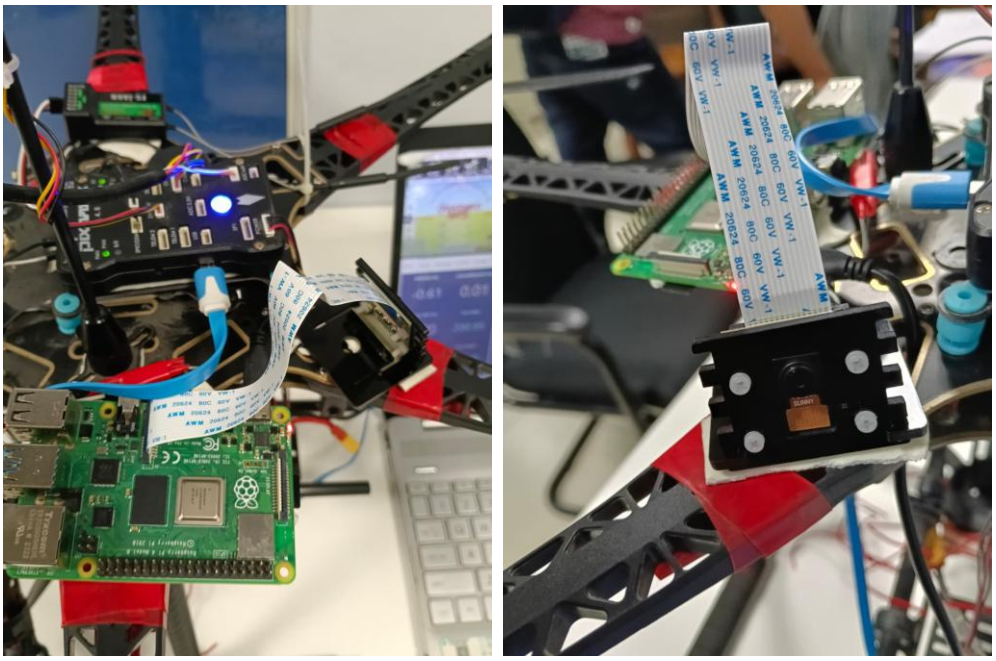


Figure 10.1: Integrated Drone-Based Surveillance System

10.2 End-to-End Workflow

The end-to-end workflow of the developed surveillance system begins with capturing the image and ends with detecting the objects from the captured image using the AI model. The steps of the complete workflow have been discussed below.

Step 1 – Capturing the Image from Drone

The Raspberry Pi Camera Module V3 captured the image of the surrounding environment. The captured image was stored in the Pi as the input for the model.

Step 2 – Providing Image Input Raspberry Pi

The captured image was provided as input to the raspberry pi 4 which acted as an edge-computing module. The input image was given to the YOLO11n model for processing and object detection.

Step 3 – Loading the Weights of YOLO11n in Raspberry PI 4

The **YOLO11n** model was loaded in the raspberry pi using the ultralytics YOLO framework. The model then processed the camera image captured from the Raspberry Pi Camera Module V3.

Step 4 – Detecting Objects in the Image

YOLO11n detected objects in the image and provided the class labels and confidence for each of the detected objects in the image.

Step 5 – Detecting the Output of the Model

The bounding boxes of the detected objects were displayed with their class labels and confidence. The developed system could able to detect objects such as a person and bus from the image.

Step 6 – Edge Based Model Inference

The inference of the model was carried out using the CPU of Raspberry Pi. It was observed that the trained model can be deployed to carry out the prediction on the images using the edge computing raspberry pi without using a desktop PC.

The developed complete end-to-end workflow connects the camera image capturing module, raspberry pi, model inference, and object detection from the image. Thus the developed system can be used for drone-based video surveillance.

10.3 Integrated Hardware Setup

The final arrangement of the developed hardware involves the key elements of the created drone-based surveillance system. The S550 hexacopter is used as the carrier for the payload, while the flight control is managed by the Pixhawk flight controller. The main on-board processor is represented by the Raspberry Pi 4 computer needed to run the image analysis algorithm. The camera was mounted on the hexacopter through the CSI interface and is used to acquire the necessary visual data for processing.

Thus, the hardware part consists of the following nodes:

- **S550 Hexacopter –** carrier for flight and payload;
- **Pixhawk Flight Controller –** flight management module;
- **Raspberry Pi 4 –** edge computing platform for the image processing algorithm;
- **Raspberry Pi Camera Module V3 –** imaging device;
- **GPS Module –** positioning and navigation sensor;
- **Radio Transmitter and Receiver –** modules for flight control;
- **Brushless Motors and ESCs –** propulsion system;
- **Battery –** power supply for the drones and hexacopter.

The Raspberry Pi 4 computer and camera were used to create an embedded vision system, while the flight management system was constructed using the Pixhawk flight controller and other

hardware components. The constructed hardware platform is capable of providing the basis for the concept demonstrated in the following section.



Figure 10.2: Integrated Hardware Setup of the Drone-Based Surveillance System

10.4 End-to-End Demonstration

The integrated system was demonstrated by combining the drone-based image acquisition and onboard AI inference components developed during the project. The Raspberry Pi Camera Module V3 was used to capture an image, which was then processed by the YOLO11n model deployed on the Raspberry Pi 4.

During the demonstration, the captured image was successfully provided to the YOLO11n inference pipeline running on the Raspberry Pi. The model processed the input image and generated object detection results in the form of bounding boxes, class labels, and confidence scores.

The demonstration confirmed the complete operation of the developed system, from image acquisition to AI-based object detection on the onboard computing platform. The detection output obtained during the Raspberry Pi experiments demonstrated the practical operation of the deployed YOLO11n model.

The end-to-end demonstration therefore verified the integration of the major system components and established the feasibility of performing AI-based visual analysis using an embedded Raspberry Pi platform.

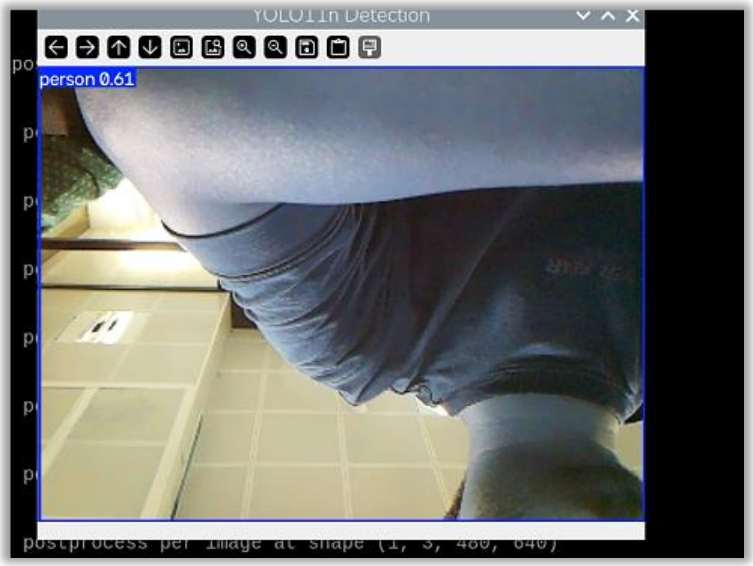


Figure 10.3: End-to-End Demonstration of YOLO11n Object Detection Using Raspberry Pi Camera Module V3

10.5 Overall Project Outcome

The project successfully established an integrated drone-based surveillance system combining aerial image acquisition, embedded computing, and AI-based object detection. The S550 hexacopter platform, Pixhawk flight controller, Raspberry Pi 4, and Raspberry Pi Camera Module V3 were brought together to provide the hardware foundation for the developed system.

The YOLO11n model was trained using aerial imagery and subsequently deployed on the Raspberry Pi 4 for edge-based inference. The integrated setup successfully captured images through the Raspberry Pi Camera Module V3 and processed them using the deployed YOLO11n model to generate object detection results with class labels and confidence scores.

The project therefore demonstrated the practical integration of drone hardware, camera-based image acquisition, embedded computing, computer vision, deep learning, and edge AI within a single surveillance-oriented platform. The developed system provides a foundation for further development of intelligent aerial surveillance applications and future improvements in real-time edge inference.

Overall, the project objectives related to system development, AI model deployment, camera-based image acquisition, and embedded object detection were successfully demonstrated through the integrated experimental setup.

CHAPTER 11

CHALLENGES AND ENGINEERING OBSERVATIONS

11.1 Hardware Integration Challenges

The integration of the drone hardware and the embedded vision components involved numerous challenges. The S550 hexacopter, Pixhawk flight controller, raspberry pi 4, and raspberry Pi Camera Module V3 module needed to come together into a unified whole, with minimal disruption to the flight control capabilities and computer vision functions.

The most significant challenges were related to the integration of the raspberry pi and camera into the hexacopter. The camera had to be placed in a location that allows it to capture the necessary field of view. Similarly, the raspberry pi had to be placed in a location that minimizes interference with the flight controller's hardware while allowing the camera to operate normally.

The integration involved connecting the **camera, Raspberry Pi, and flight controller**, along with proper **cable management, power supply, and mounting** to ensure a secure and accessible setup.

The hardware integration process was an eye-opener in terms of understanding the level of detail and precision needed to make a working drone-based intelligent surveillance system. The processes taught multiple lessons related to the construction, configuration, integration, and testing of a computer vision system based on drones. In particular, the integration process showed how important it is to consider the interactions between all components when developing such systems.

11.2 Raspberry Pi Connectivity and Remote Access Challenges

While working on the Raspberry Pi deployment, ensuring connectivity was essential for establishing communication, transferring model files, and performing remote testing procedures. In particular, the Raspberry Pi unit was deployed and utilized as a remote computer via networks to manage the required model files, programs, and tests without direct physical interaction with the device via a monitor and keyboard.

In particular, one of the primary challenges during the Raspberry Pi model deployment was to gain access to the device through networks. In this regard, the Raspberry Pi unit was addressed via its network address to establish connections. In this regard, remote access was critical for deploying the project and performing all necessary tests, including transferring the required fine-tuned **best.pt** model from the Windows computer to the target Raspberry Pi machine.

The remote access to the device significantly simplified the model deployment procedure since all necessary files, including the model files, Python programs, and test images, could be directly organized and transferred from the computer to the target machine. At the same time, the connection had to be established every time the Raspberry Pi device was used to ensure the possibility of file transfer or executing terminal commands.

Overall, the model deployment process showed the significance of proper network configuration and remote access to the target machine in embedded AI application development. At the same time, the remote model deployment and testing procedures also gave a useful insight into working with such devices in general and managing them remotely in particular.

11.3 Model Deployment and Dependency Challenges

The deployment phase of the trained YOLO11n model from the Windows environment to the raspberry pi involved numerous challenges relating to model file management and software environment requirements. Initially, the inference-ready **best.pt** model file was transferred to the raspberry pi from the Windows machine using secure copy (SCP). The file was then moved to the model folder to perform inference.

An important aspect of the model deployment process was ensuring that the raspberry pi had the appropriate environment setup for loading and deploying the YOLOv11 model. The inference environment had to include important packages such as **Ultralytics, PyTorch, OpenCV, and NumPy.** The versions of these packages in the raspberry pi had to be checked and updated where necessary before model deployment.

Finally, it was necessary to ensure that the model file path, Python inference program path, and test image path were correctly set up. This would help prevent any issues that may arise due to the model not being deployed correctly. In general, the model deployment process involved numerous challenges associated with file management and software preparation.

In summary, this project involved multiple steps in the development and deployment of an edge-AI application using the YOLOv11 model for object detection. Firstly, the project involved training a state-of-the-art YOLOv11 model using a custom data set. The model training process encountered numerous challenges in terms of data preparation, model setup, and performance optimization. Secondly, the model deployment phase was equally challenging as it involved deploying the trained model on an embedded platform. The model deployment process involved several challenges in terms of model file management and environment setup for model inference.

11.4 Raspberry Pi Computational Constraints

The attempt to run the YOLO11n model directly on the Raspberry Pi illustrated the computational limitations of this embedded edge-computing device. While the YOLO11n model is lightweight enough to run on such a device, the time taken by the CPU was comparatively long for each input image.

Specifically, during testing, the Raspberry Pi processed the YOLO11 model with an **inference time of approximately 1450.6 ms,** and **17.5 ms** and **5.5 ms** for pre-processing and post-processing, respectively. Overall, the frame rate for the selected model on the Raspberry Pi would be approximately **0.68 FPS**.

The results illustrate that while it is possible to perform the YOLO11n object detection model without a desktop computer, the computational limitations of the Raspberry Pi prevent it from being able to process video at a reasonably high frame rate. Thus, the device is only appropriate for image-based analysis or for lower-resolution video processing. For real-time processing of higher-resolution videos, one would need to optimize the model further, potentially by reducing the resolution, selecting a more appropriate model format, or using more powerful edge-computing devices.

The task provided a useful insight into the challenges of running computationally intensive tasks, such as deep learning, on embedded devices. While the Pi had enough power to execute a relatively lightweight model, its performance was limited in terms of raw power. This characteristic reduces the device's usefulness for real-time processing. However, the task also highlighted that some optimizations can be made to improve the inference time without reducing the model's usefulness.

11.5 Drone Flight and Validation Challenges

The flight-validation stage included the drone platform, the flight controller, the propulsion system, and the onboard electronics. Prior to the flight tests, the primary control parameters and sensors had to be commissioned in order to ensure stable operation of the aircraft.

The practical aspect of this stage included ensuring the correct functioning of the whole system with all the necessary hardware components. In particular, the addition of the Raspberry Pi and the camera created additional complexity since it had to be physically installed on the drone and connected to the power supply and the flight-controller electronics.

Finally, the flight validation included observing the behavior of the aircraft during actual flight in order to ensure that all flight modes, sensors, radio control, and safety mechanisms function correctly. If any of them did not behave as expected, it had to be adjusted prior to proceeding to the next stage.

In this stage, I gained valuable experience with the transition from testing individual systems to working with an entire complex. I saw the importance of each step in this process, as well as the necessity of testing each change and adjustment prior to proceeding to the next stage. This process is critical for ensuring the safe operation of the aircraft, as well as the proper integration of embedded vision systems. Thus, this stage allowed me to gain practical experience both with testing hardware components and with system-level operations in a real-world setting.

11.6 Engineering Lessons Learned

I believe that the work on the drone surveillance system was educational in terms of seeing the synergistic nature of the engineering fields involved and learning the practical aspects of each of them. In particular, the project helped me understand the importance of the collaboration between hardware, software, embedded Linux, computer vision, and AI components in creating an efficient system.

First and foremost, this project taught me that the integration and testing of separate systems is essential in developing products such as the drone surveillance system. The separate aspects of the system needed to be checked and debugged before the whole product was ready for the final testing and quality control. This step helped determine the possible configuration and integration issues and resolve them before they could cause bigger problems down the line.

I also expanded my knowledge and gained more practical experience in **embedded Linux, computer vision, and remote system management** through this course. In particular, working on the Raspberry Pi unit taught me a lot about networking and managing remote systems and devices. I also learned how to use Python to transfer files to the remote server and create and manage Python environments. Moreover, I became more familiar with the process of installing various dependencies and packages necessary for the Python environment on the remote machine and saw how the AI model runs on the RPi.

Finally, this project gave me insight into the difference between developing an AI model on a powerful computer and using the model on a resource-limited embedded device. In particular, I saw how working with actual hardware was different from working with a computer that had

sufficient space and power. Thus, the project was also educational in terms of my understanding of the technical aspects of embedded AI development.

In conclusion, I believe that the given project was an incredibly valuable exercise because it allowed me to see how different aspects of embedded system development, such as hardware, software, networking, and computer vision, come together in practical applications, such as the drone surveillance system. Additionally, I was able to enhance my understanding of the importance of testing and debugging separate components and the overall system as well as practical experience with remote server management and embedded computer vision.

CHAPTER 12

FUTURE SCOPE

The system we made shows that it is possible to use a UAV platform with image processing and YOLO11n-based object detection in one. To make the UAV system better we can work on making it more autonomous so it can do things on its own. We can also try to make it work faster in time make the flights more reliable and make the UAV system last longer when it is operating. The UAV system needs to be improved in these areas to make it really useful.

12.1 Autonomous Flight and GPS Navigation

The **Pixhawk flight controller and GPS** provide a foundation for autonomous UAV navigation. Future work can integrate the **Raspberry Pi-based perception system with Pixhawk** to enable the UAV to follow predefined GPS points with less operator involvement.

The system can further be extended to use **Raspberry Pi visual information** for mission decisions. This would require communication between the Raspberry Pi and Pixhawk, followed by ground and flight testing.

The mission could look something like this:

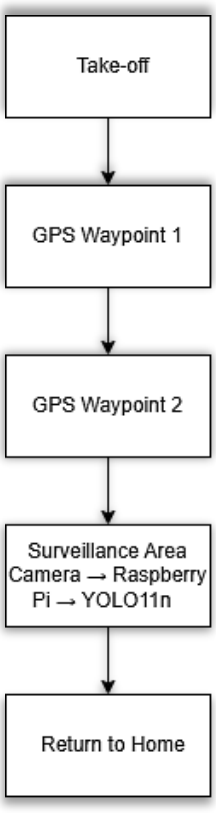


Figure 12.1: Proposed Autonomous UAV Mission and AI-Based Surveillance Flow

As mentioned in this chapter, the camera and the AI-based computer vision system built around the Raspberry Pi would work in tandem with the flight controller navigation system. This means that the images taken by the drone during the mission could go through the YOLO11n-based computer vision system right on the drone, thus creating an autonomous AI surveillance system.

12.2 Real-Time Aerial Inference and Live Video

The system we have now does something cool. It uses the Raspberry Pi 4 and the Raspberry Pi Camera Module V3 to detect objects in pictures.

We are thinking about what we can do. Maybe we can make it work with videos of just pictures. This would be really useful for things like monitoring what is happening from the air in time.

The Raspberry Pi 4 system could look at some of the frames from a video and detect objects in them. At the time it could send live video or video that has been processed to a station on the ground. This would let someone watch what the drone is doing and see what objects it has found while it is flying.

If we want to process video all the time the Raspberry Pi 4 system will need to be able to handle it. This means we have to be careful about how frames, per second it can handle and what resolution we use. We do not want it to use much memory or power.

12.3 Model Optimization for Edge Deployment

The time it takes to make a decision on the Raspberry Pi is **1.45 seconds and 1.64** seconds for each image. This is too slow. The system needs to be improved so it can make decisions faster when it is on the UAV.

To make the system better we can look at using models making images smaller reducing the size of the models and using special hardware to make it faster. The goal is to make the system make decisions faster and use power while still being able to detect things correctly.

We need to test the system using real world tests, such, as how accurate it is how long it takes to make a decision how much memory it uses, how much of the CPU it uses and how much power it uses. This will help us figure out if the Raspberry Pi system is good enough to be used on a UAV all the time.

12.4 Improved Flight Stability and Reliability

The aerial platform can be improved by increasing its **stability and reliability during image capture**. Camera vibration, UAV movement, wind, and payload changes can reduce image quality and affect object detection performance.

Future improvements can include **better camera vibration isolation, flight-controller tuning and calibration, balanced propulsion components, and proper payload distribution**. These changes can provide smoother flight and clearer images, improving the consistency of the overall object detection system.

12.5 Autonomous Surveillance Missions

The current system can be further developed into an autonomous surveillance platform capable of performing assigned missions with minimal operator involvement.

A possible future mission could be:

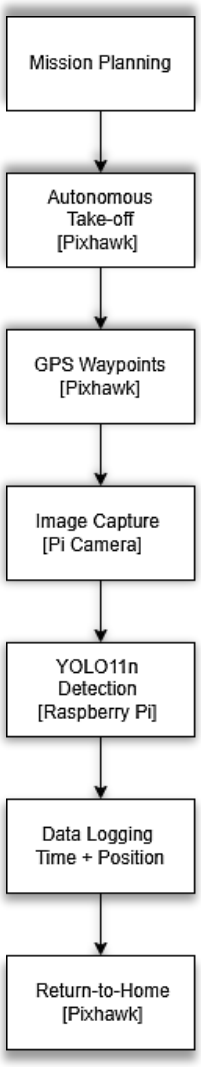


Figure 12.2: Proposed Autonomous Surveillance Mission Workflow

Detected objects could be recorded together with the corresponding timestamp and UAV position. With further development, the UAV could also change its observation position or perform a predefined action when a particular target is detected.

This would extend the system from onboard object detection to autonomous mission execution, while retaining the Pixhawk for flight-control functions and the Raspberry Pi for onboard AI processing.

12.6 Autonomous Landing, Docking and Wireless Charging

One of UAV's major limitation is its **limited flight time**, restricting the duration of continuous surveillance. One plausible development would be to design an autonomous docking station, with which the UAV could automatically land, recharge its battery, and resume the mission without requiring operator intervention.

First, the UAV would use its GPS to navigate towards the area in which the docking station is located. Then, to perform the landing, either another visual marker or a more advanced precision landing method would have to be used to navigate the UAV to the docking station. When the UAV lands, it would then be able to begin recharging.

The S550's best option would be to create a docking station on the ground, rather than to attempt to dock and charge while in flight. This would probably take the form of a wireless battery charging coil on the landing pad, with another coil attached to the UAV, which would have to be properly aligned with the other coil in order to begin charging.

Alternatively, if the S550 were to develop an automatic battery swapping system, this would drastically reduce the downtime between missions, allowing for more hours of total surveillance time. The downside would be the mechanical and electrical complexity such a system would require.

The development of such a system would have to be a gradual process, beginning from simple autonomous landing procedures, then progressing to systems which could automatically begin charging when on the **docking station,** and then working towards more complicated methods of refueling the UAV.

CHAPTER 13

CONCLUSION

13.1 Conclusion

The project **"Intelligent Electronic Surveillance with Camera-Integrated Drones"** successfully established and demonstrated an embedded AI-based surveillance system by integrating a drone platform, flight-control system, camera, and edge-computing device. The developed platform combined the **S550 hexacopter, Pixhawk flight controller, Raspberry Pi 4, and Raspberry Pi Camera Module V3** to provide an aerial platform for image acquisition and onboard processing.

During the project, the Raspberry Pi Camera Module V3 was successfully configured and tested for image acquisition. The **YOLO11n** deep learning model was studied, trained, fine-tuned, and deployed on the Raspberry Pi 4 for object detection. The **VisDrone dataset** was used as part of the model-development process, and the fine-tuned model was transferred from the training environment to the Raspberry Pi for edge-based inference.

The deployed system was successfully used to perform object detection on camera-captured images as well as on a live camera stream. The detection output provided object labels, bounding boxes, and confidence scores, demonstrating the practical use of onboard AI for image-based surveillance. The live-stream demonstration further established that the Raspberry Pi could perform continuous camera-based AI inference without requiring a desktop computer for the inference process.

The edge-inference evaluation also provided an understanding of the computational requirements of running YOLO11n on the Raspberry Pi CPU. The measured inference time demonstrated the feasibility of embedded object detection while also indicating the need for further optimization when higher frame rates are required.

Overall, the project successfully brought together **UAV hardware, flight control, camera-based image acquisition, deep learning, computer vision, and edge computing** into a functional surveillance prototype. The developed system provides a practical foundation for further development toward autonomous aerial surveillance, optimized real-time inference, GPS-based missions, improved flight stability, and automated drone operation.

REFERENCES

[1] A. Yadav, M. Fujita, and Binod Kumar, “Lightweight DL-based Drone Image Classification for Surveillance Applications,” in *Proc. 21st Int. SoC Design Conf. (ISOCC)*, Sapporo, Japan, Aug. 19–22, 2024, doi: 10.1109/ISOCC62682.2024.10762201.

[2] Ultralytics, “Ultralytics YOLO Documentation,” [Online]. Available: Ultralytics Documentation. [Accessed: Aug. 17, 2026].

[3] K. Pandya, “VisDrone Dataset,” Kaggle, [Online]. Available: VisDrone Dataset. [Accessed: Aug. 17, 2026].

[4] Raspberry Pi Ltd., “Raspberry Pi,” [Online]. Available: Raspberry Pi. [Accessed: Aug. 17, 2026].

[5] ArduPilot, “Pixhawk Overview,” ArduPilot Copter Documentation, [Online]. Available: Pixhawk Overview. [Accessed: Aug. 17, 2026].

[6] ArduPilot, “Mission Planner Documentation,” [Online]. Available: Mission Planner Documentation. [Accessed: Aug. 17, 2026].

[7] PyTorch, “PyTorch Documentation,” [Online]. Available: PyTorch Documentation. [Accessed: Aug. 17, 2026].

[8] OpenCV, “OpenCV Documentation,” [Online]. Available: OpenCV Documentation. [Accessed: Aug. 17, 2026].

[9] Robu.in, “Electronic Components and Robotics Store,” [Online]. Available: Robu.in. [Accessed: Aug. 17, 2026].

[10] Pro Bots India, “Electronic Components and Robotics Products,” [Online]. Available: Pro Bots India. [Accessed: Aug. 17, 2026].

[11] Amazon India, “Amazon.in,” [Online]. Available: Amazon India. [Accessed: Aug. 17, 2026].

[12] diagrams.net, “Flowchart Maker & Online Diagram Software,” [Online]. Available: diagrams.net. [Accessed: Aug. 17, 2026].

APPENDICES

Appendix A — Important Code and Inference Commands

This appendix presents the principal code and commands used during model deployment and inference on the Raspberry Pi.

A.1 YOLO11n Model Loading

The YOLO11n model was loaded in the Raspberry Pi Python environment using the Ultralytics library.

```
from ultralytics import YOLO
model = YOLO("yolo11n.pt")
```

A.2 Image-Based Inference

The captured image **test.jpg** was processed using the loaded model. The save=True option was used to save the detection output.

```
results = model("test.jpg", save=True)
```

A.3 Inference Completion Verification

The completion of the inference process was verified using:

```
print("Inference Completed Successfully")
```

A.4 Complete Inference Sequence

```
from ultralytics import YOLO
model = YOLO("yolo11n.pt")
results = model("test.jpg", save=True)
print("Inference Completed Successfully")
```

Appendix B — YOLO Training and Verification Commands

B.1 Initial YOLO11n Training

The initial YOLO11n model was trained for 20 epochs using the prepared aerial-surveillance dataset.

Command:

```
yolo detect train model=yolo11n.pt data=Dataset\data.yaml epochs=20 imgsz=640 batch=8 device=cpu workers=0 project=runs\detect name=DroneSurveillance_YOLO-2
```

B.2 YOLO11n Fine-Tuning

The **best.pt** checkpoint from the initial training was subsequently used for another 20 epochs of fine-tuning.

```
yolo detect train model="runs/detect/DroneSurveillance_YOLO-2/weights/best.pt" data="Dataset/data.yaml" epochs=20 imgsz=640 batch=8 device=cpu workers=0 project="runs/detect" name="DroneSurveillance_Finetune"
```

B.3 Model Inference

A test image was processed using:

Command:

```
from ultralytics import YOLO
model = YOLO("yolo11n.pt")
model("test.jpg", save=True)
```

B.4 Detection Output Verification

The generated detection images were located using:

Command: `find runs -name "*.jpg"`

The recorded output included:

runs/detect/predict/bus.jpg

B.5 Detection Directory Verification

Command: `ls runs/detect`

Recorded directories included:

- predict
- predict-2
- predict-3

The detection labels were examined using:

Command:

ls runs/detect/predict-3

cat runs/detect/predict-3/labels/bus.txt

The recorded YOLO-format label data were:

5 0.493846 0.443395 0.978312 0.462004

0 0.914061 0.589620 0.171409 0.447964

0 0.177064 0.603591 0.237024 0.467244

0 0.350354 0.587563 0.149884 0.418234

0 0.0424625 0.661306 0.0848661 0.292815

Appendix C — Raspberry Pi Setup and Deployment

C.1 Camera Verification

The Raspberry Pi Camera Module V3 was checked using:

Command: rpicam-hello

The camera initialized successfully and the preview was verified.

C.2 Image Capture

A test image was captured using:

Command: rpicam-still -o test.jpg

The captured file was verified using:

Command: ls -lh test.jpg

Recorded file information:

-rw-rw-r-- 1 amanaman 909K Jul 20 11:00 test.jpg

C.3 Python Environment

The Raspberry Pi Python environment was checked using:

- python3
- Recorded Python version:
- Python 3.13.5

C.4 Project Directory

The deployed project was maintained in:

Command:

```
cd ~/Desktop/YOLO_Model
```

```
ls -lh ~/Desktop/YOLO_Model
```

The recorded files included:

File	Approx. Size
best.pt	5.2 MB
yolo11n.pt	5.4 MB
pi_camera_yolo.py	1.2 KB
test.jpg	82 KB

Table C.4 — Deployed Project Files and Approximate Sizes

C.5 Model Transfer

The fine-tuned best.pt model was transferred from the Windows development system to the Raspberry Pi using SCP.

Command:

```
scp "C:\Users\amans\Desktop\Drone_AI_Project\runs\detect\runs\detect\DroneSurveillance_Finetune\weights\best.pt" aman@10.38.4.1:/home/aman/Desktop/YOLO_Model/
```

The transferred model was subsequently verified on the Raspberry Pi:

Command:

```
cd ~/Desktop/YOLO_Model
```

```
ls -lh
```

The deployed **best.pt** file was approximately **5.2 MB**.

C.6 Camera Inference Program

The Raspberry Pi camera and YOLO11n model were integrated through a Python program to capture an image from the Raspberry Pi Camera Module V3 and perform object detection using the trained model. The program used the captured image as input and saved the resulting detection output.

Python Code:

```
from picamera2 import Picamera2

from ultralytics import YOLO

import time

# Initialize camera

picam2 = Picamera2()

picam2.configure(

    picam2.create_still_configuration(

        main={"size": (640, 480)}

    )

)

picam2.start()

# Allow the camera to initialize

time.sleep(2)

# Capture image

image_path = "test.jpg"

picam2.capture_file(image_path)

# Load YOLO11n model

model = YOLO("best.pt")

# Perform object detection

results = model(image_path, save=True)
```

```
print("Image captured successfully")

print("Inference Completed Successfully")

picam2.stop()
```

The program was executed on the Raspberry Pi using:

Command:

```
cd ~/Desktop/YOLO_Model

python3 pi_camera_yolo.py
```

The captured image was saved as **test.jpg,** after which the YOLO11n model performed inference and saved the corresponding detection result.

Appendix D — Model Training Configuration

D.1 Model Configuration

Table D.1: YOLO11n Model Training Configuration

Parameter	Value
Model	YOLO11n
Task	Object Detection
Dataset	VisDrone-based aerial surveillance dataset
Initial Training	20 epochs
Fine-Tuning	20 epochs
Total Training Epochs	40 epochs
Image Size	640 × 640 pixels
Batch Size	8
Processing Device	CPU
Workers	0
Framework	Ultralytics YOLO
Python Environment	Python virtual environment

D.2 Initial Training

The initial training stage used 20 epochs, with an image size of 640 × 640, batch size of 8, CPU processing, and 0 workers. The recorded number of iterations per epoch was 809, resulting in:

809 × 20 = 16,180 iterations

D.3 Initial Training Losses

Table D.2: Final-Epoch Training Losses from Initial YOLO11n Training

Loss	Value
Box Loss	1.496
Classification Loss	1.079
DFL Loss	0.912

D.4 Fine-Tuning Configuration

The previous best.pt checkpoint was used for a further 20 epochs under the same image-size, batch-size, CPU, and worker settings.

Table D.3: YOLO11n Fine-Tuning Configuration and Training Details

Parameter	Value
Starting model	Previous best.pt
Fine-tuning epochs	20
Image size	640 × 640 pixels
Batch size	8
Device	CPU
Workers	0
Output	DroneSurveillance_Finetune
Training duration	Approximately 20.139 hours
Pretrained Items Transferred	499/499

The training log confirmed that **499/499** pretrained items were successfully transferred.

D.5 Dataset Scanning Results

Table D.4: Dataset Scanning Results for the YOLO11n Training and Validation Dataset

Dataset Information	Count
Training images	6,471
Validation images	548
Background-only images	0
Corrupt images	0
Validation annotations	38,759 object instances

Duplicate labels identified during dataset preparation were automatically removed.

D.6 Fine-Tuning Final-Epoch Losses

Table D.5: Final-Epoch Training Losses from YOLO11n Fine-Tuning

Loss	Value
Box Loss	1.445
Classification Loss	1.010
DFL Loss	0.9006

D.7 Validation Performance

The final validation results were:

Table D.6: Validation Performance of the Fine-Tuned YOLO11n Model

Metric	Value
Precision	0.389
Recall	0.319
mAP@50	0.281
mAP@50–95	0.157

D.8 Model Complexity

The final YOLO11n model contained approximately 2.58 million parameters and 6.3 GFLOPs.

The training process generated:

- best.pt
- last.pt

The best.pt checkpoint was selected for deployment. Its recorded size was approximately **5.4 MB** after optimizer information was removed and approximately **5.2 MB** after deployment to the Raspberry Pi.

Appendix E — Representative Detection Results

E.1 Raspberry Pi Camera Module V3 Detection

The Raspberry Pi Camera Module V3 image test.jpg was processed using YOLO11n.

Recorded result:

- image 1/1 /home/aman/test.jpg: 480x640 1 person, 1450.6ms
- Speed: 17.5ms preprocess, 1450.6ms inference,
- 5.5ms postprocess per image at shape (1, 3, 480, 640)

Results saved to /home/aman/runs/detect/predict-2

The detected object was:

person 0.59

The corresponding detection result showed:

person 0.59

E.3 Bus Image Detection

The **bus.jpg** image was also processed to verify detection performance.

Recorded result:

- image 1/1 /home/aman/bus.jpg: 640x480
- 4 persons, 1 bus, 1642.0ms
- Speed: 30.8ms preprocess, 1642.0ms inference,
- 54.2ms postprocess per image at shape (1, 3, 640, 480)

Results saved to /home/aman/runs/detect/predict

Inference Completed Successfully

Representative detections included:

Table E.1: Representative Bus Image Detection Results

Class	Confidence
Bus	0.94
Person	0.88
Person	0.86
Person	0.62

The corresponding YOLO-format labels were:

- 5 0.493846 0.443395 0.978312 0.462004
- 0 0.914061 0.589620 0.171409 0.447964
- 0 0.177064 0.603591 0.237024 0.467244
- 0 0.350354 0.587563 0.149884 0.418234
- 0 0.0424625 0.661306 0.0848661 0.292815

Appendix F — Drone Parts List

Table F.1 — Drone Parts List

S. No.	Component	Quantity
1	S550 Hexacopter Frame	1
2	Pixhawk 2.4.8 Flight Controller	1
3	Brushless Motors	6
4	Electronic Speed Controllers (ESCs)	6
5	Propellers	6
6	GPS Module	1
7	Radio Transmitter	1
8	Radio Receiver	1
9	LiPo Battery	1
10	Power/Voltage Regulation Components	1

F.2 Embedded Vision System

Table F.2 — Embedded Vision System

S. No.	Component	Purpose
1	Raspberry Pi 4	Edge computing and AI inference

2	Raspberry Pi Camera Module V3	Image acquisition
3	Camera CSI Interface	Camera-to-Raspberry Pi connection
4	MicroSD Storage	Raspberry Pi operating system and software

F.3 Component Procurement/Reference Sources

The hardware components used or referenced during the project were sourced/referenced through:

- Robu.in
- ProBots India
- Amazon India

These sources are included for component procurement/reference and are not treated as technical documentation sources.

Appendix G — Week-Wise Internship Progress

The internship work from **01 June 2026, to 20 July 2026** progressed from embedded systems and autonomous navigation studies to computer vision, deep learning model development, Raspberry Pi based edge AI, UAV configuration, live detection, flight testing and system level integration. The major work completed during each week is summarized below.

G.1 Week 1 — Embedded Systems and Autonomous UAV Path Planning

01–07 June 2026

The first week focused on **Embedded Systems and Autonomous UAV Path Planning**. I studied the **DSL-2 kit lab manual** from STMicroelectronics based on the **STM32F4 series** and configured **Keil µVision and MDK**. The study covered ARM assembly, GPIO, timers, ADC, UART, I²C, external interrupts, sensor interfacing, and embedded programming.

In parallel, I studied autonomous UAV surveillance concepts, including mapping, localization, obstacle avoidance, and path planning. I prepared the report **"Autonomous Surveillance Drone Path Planning: Initial Study"** and implemented a basic grid-based representation with the **A* algorithm** to understand autonomous path planning.

G.2 Week 2 — Autonomous Navigation, Sensors and Sensor Fusion Study

08–14 June 2026

The second week focused on **autonomous navigation and robotic perception**. I studied autonomous vacuum-robot navigation, including sensing, localization, mapping, obstacle avoidance, **SLAM**, and path planning, and prepared a separate technical report.

I also studied **LiDAR, encoders, IMU/MPU6050, and cameras** for robotic perception; LiDAR-based mapping; **A* and Dijkstra** algorithms; occupancy-grid mapping; SLAM; and **Kalman filtering** for sensor fusion and localization. The study also considered future possibilities such as autonomous drones and wireless charging for extended operation.

G.3 Week 3 — Raspberry Pi, Camera, Python and OpenCV Environment

15–21 June 2026

The third week focused on the **Raspberry Pi Camera Module V3** and Python-based image acquisition. I implemented an image-capture script that captured images at defined intervals during drone operation. The workflow was designed to operate locally **without internet connectivity or VNC access**.

A separate live-streaming script was also implemented for real-time camera-feed transmission and monitoring. Image acquisition and basic processing were verified using **Python and OpenCV**, and the scripts, configurations, test results, screenshots, and observations were recorded as supporting evidence.

G.4 Week 4 — Dataset Analysis and YOLO Model Selection

22–28 June 2026

The fourth week focused on preparing the **AI-based object-detection pipeline**. I downloaded and examined the **VisDrone-based aerial dataset**, including its object classes, annotations, image structure, and suitability for aerial object detection.

Different YOLO variants were studied in terms of model size, computational requirements, detection capability, and deployment suitability. Based on the available hardware and Raspberry Pi inference requirements, **YOLO11n** was selected. The **Python, PyTorch, Ultralytics, and OpenCV** environments were configured for model development and training.

G.5 Week 5 — YOLO11n Training and Deep-Learning Development

29 June–05 July 2026

The fifth week concentrated on **YOLO11n model development and training**. The prepared dataset was configured for object detection and an initial **20-epoch training** was performed. Training losses, validation behaviour, model outputs, and relevant logs were monitored and recorded.

I also studied model evaluation using **Precision, Recall, mAP@50, and mAP@50–95** and examined the trained checkpoints to select the best model for further fine-tuning. The complete workflow from dataset preparation and configuration to training and validation was documented for edge deployment.

G.6 Week 6 — Fine-Tuning, Edge AI and UAV Configuration

06–12 July 2026

The sixth week involved **model refinement, edge-AI deployment, and UAV configuration**. The selected best.pt model from the initial training was fine-tuned for a further **20 epochs**, followed by validation and documentation of the resulting performance.

The fine-tuned model was transferred to the **Raspberry Pi 4** and configured with the required **Python, PyTorch/Ultralytics, and OpenCV** environment for camera-based edge inference. I also studied the **Raspberry Pi AI HAT** as a potential hardware-acceleration approach for future edge-AI development.

In parallel, I worked with the **S550 hexacopter and Pixhawk flight-control system**. Accelerometer and compass calibration, receiver/transmitter setup, flight modes, failsafe configuration, motors, propellers, power system, battery, and payload arrangement were checked and calibrated before flight testing.

G.7 Week 7 — Live AI Detection, UAV Flight Testing and System Integration

13–20 July 2026

The final week focused on **system integration and testing**. The Raspberry Pi 4, camera, and fine-tuned YOLO11n model were integrated for camera-based edge inference. I implemented and tested **live-stream object detection**, enabling camera video to be processed by YOLO and detected objects to be displayed in real time for drone-based surveillance applications.

The S550 hexacopter was first tested through **controlled low-altitude flights** to verify stability, control response, power behaviour, and basic operation. Subsequent **higher-altitude testing** was

conducted under controlled conditions. Motor, propeller, receiver/transmitter, battery, payload, and flight-control behaviour were observed, together with AI inference and image/video outputs.

The final stage involved **debugging, integration testing, documentation, and preservation of experimental evidence**, including model outputs, training results, calibration values, flight observations, photographs, screenshots, and test records. These results were organized for the final internship report and for identifying future improvements in autonomous UAV navigation, real-time edge AI, and reduced human intervention.

G.8 Overall Internship Progress

During the internship, I worked across **embedded systems, autonomous navigation, sensor fusion, computer vision, deep learning, edge AI, Raspberry Pi, UAV hardware, and system integration**. The work progressed from research and algorithm studies to **YOLO11n training and fine-tuning, Raspberry Pi deployment, live-stream detection, S550/Pixhawk configuration, controlled flight testing, and AI-based UAV surveillance integration**. Technical reports, experimental results, calibration records, photographs, screenshots, model outputs, and test observations were maintained to support the final documentation and future development.